

Topic 14

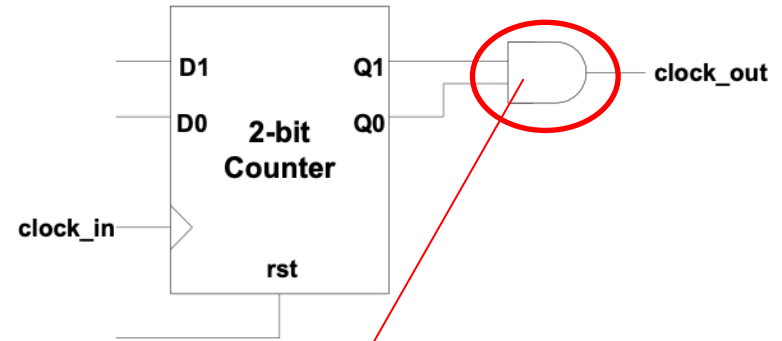
Timing Issues

Outline

- Gated Clock
- Timing requirements for FSM
- Setup time and hold time
- Asynchronous input and synchronizer
- Switch debouncing
- Hazards

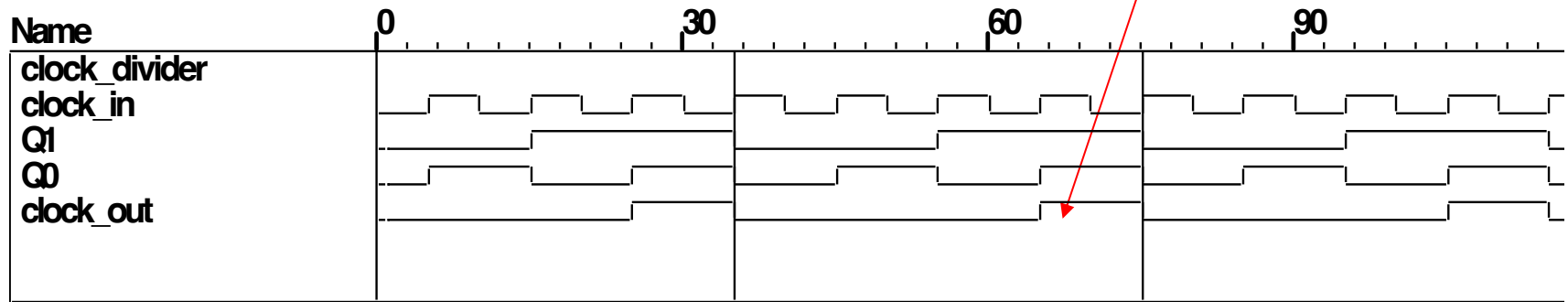
Clock Divider

- 4-fold clock divider: 2-bit binary counter
- The clock_out generates one pulse for every four input clock cycles.
 - So the frequency of the clock_in is reduced by 4 times



Ideal case, delay ignored

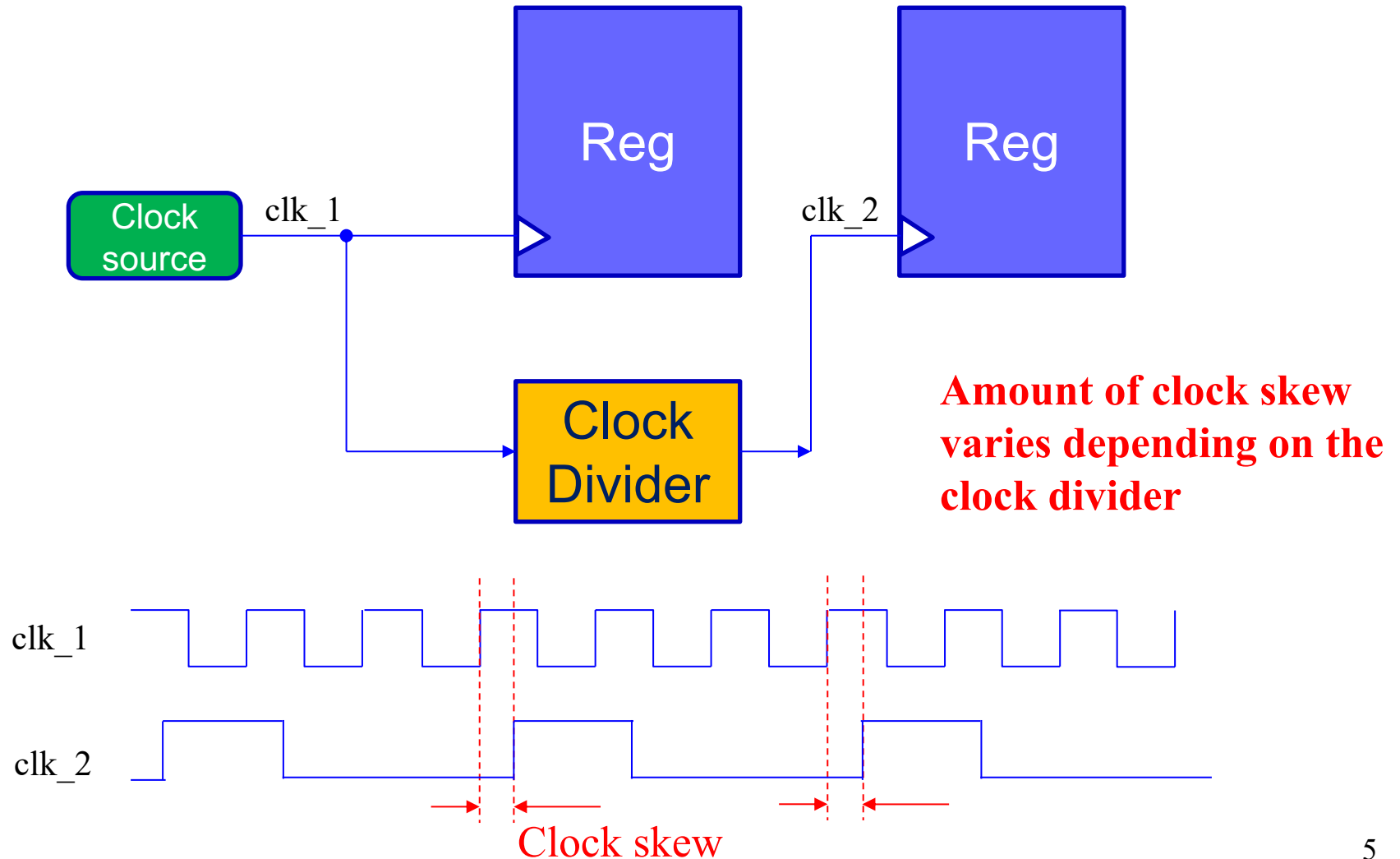
T1 35 T2 75 Tdelta 40



Gated Clock

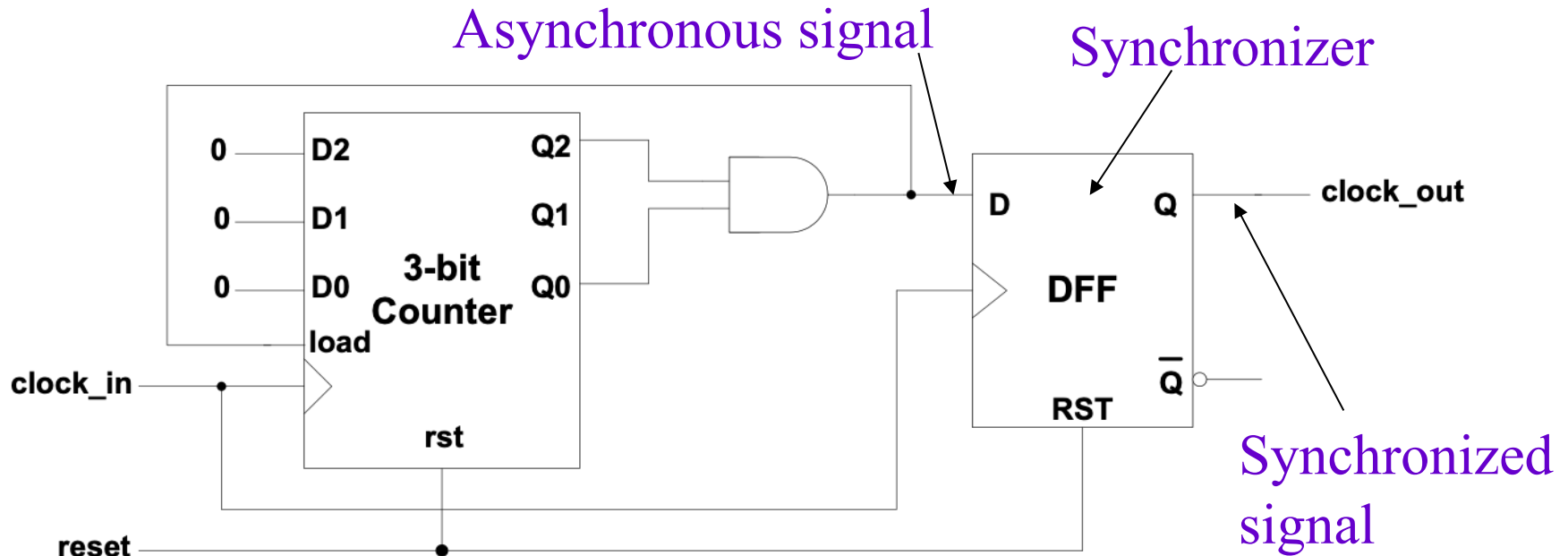
- Propagation delay of the output clock:
 - The output of the clock dividers comes from a combinational logic circuit (single AND gate or a net of gates in multiple levels)
 - The output will be delayed due to delay of the gate(s)
 - Amount of delay depends on the combinational circuit
 - This kind of output is called **gated output** or **asynchronous output** or **unregistered output**
 - If the gated output is used as a synchronizing signal (i.e. clock), the output may cause **clock skew**
 - **Clock skew:** the difference in the arrival time of clock signal between *two sequentially-adjacent registers* (wikipedia)
 - Causes synchronous circuit to behave asynchronously

Gated Clock – Causing Clock Skew



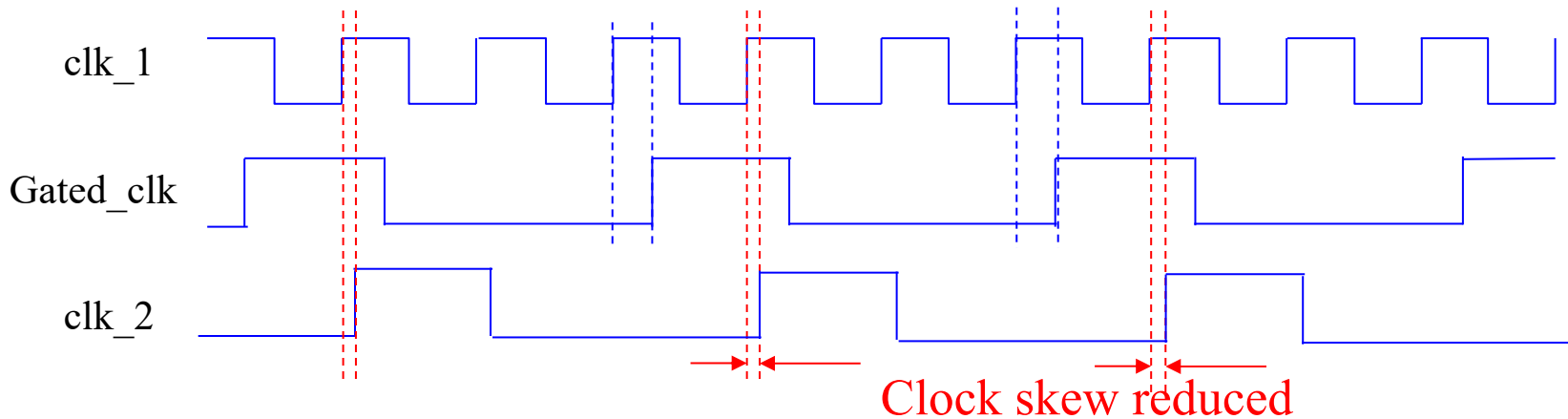
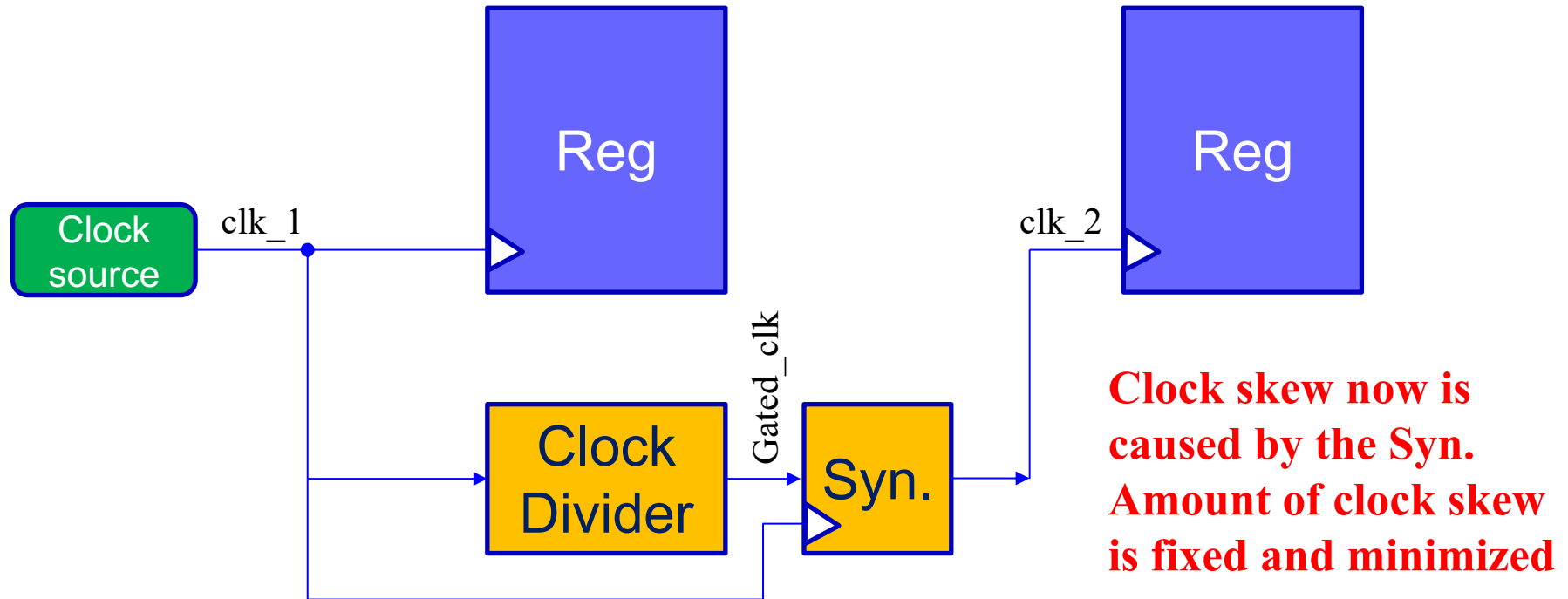
Synchronizer

- One possible solution to reduce clock skew is to synchronize the output by a D flip-flop



- Synchronizer aligns the synchronized signal with clock input
 - But delays the input asynchronous signal by up to 1 clock cycle
 - Clock skew is not completely removed, only reduced

Synchronized Clock

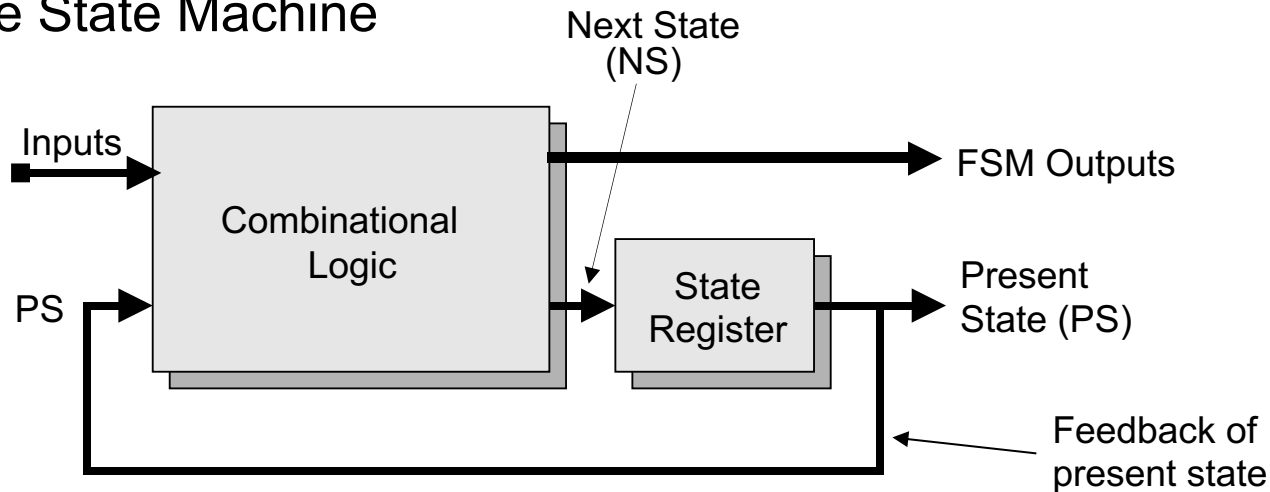


Outline

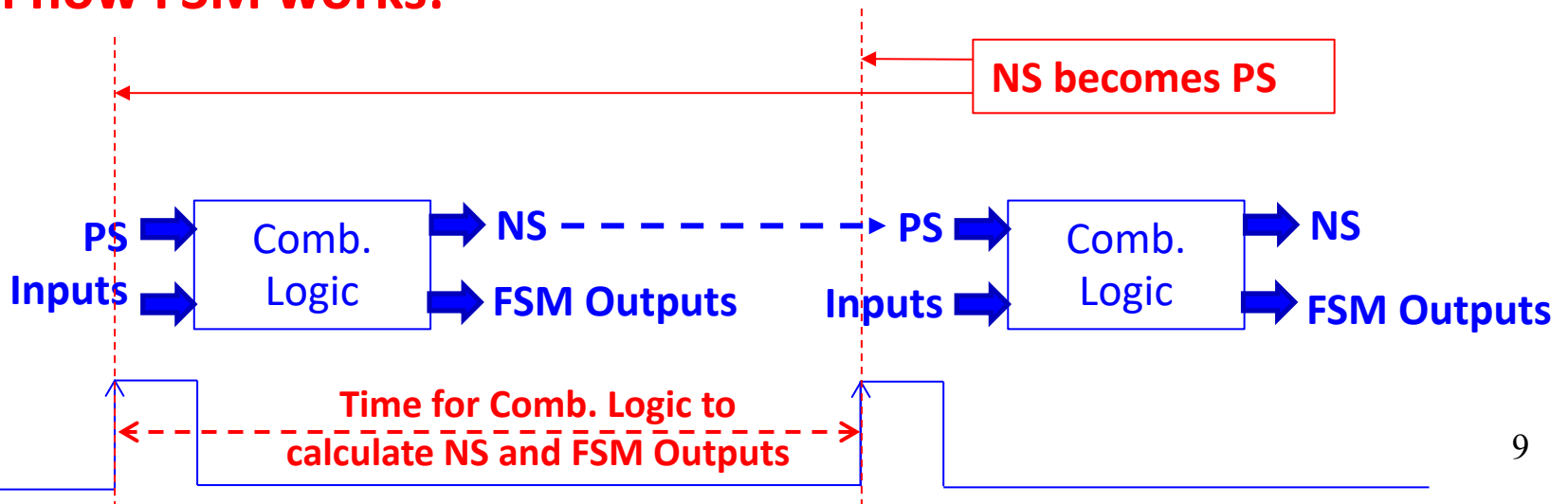
- Gated Clock
- Timing requirements for FSM
- Setup time and hold time
- Asynchronous input and synchronizer
- Switch debouncing
- Hazards

State Machine

- Finite State Machine

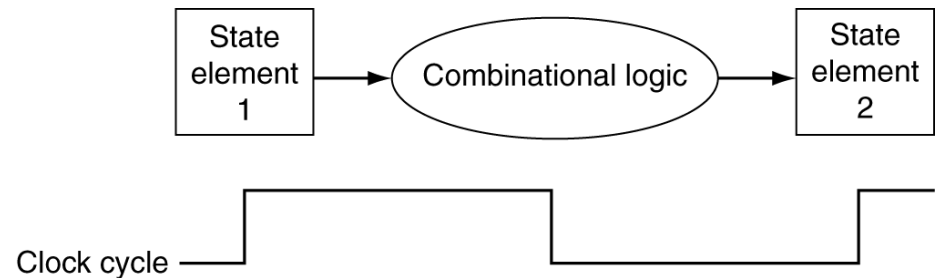
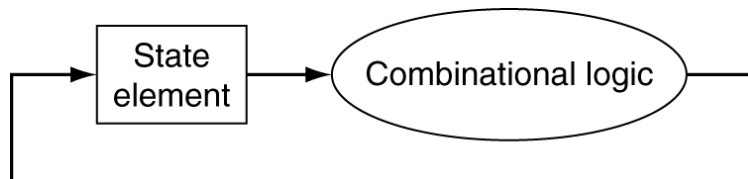


Recall how FSM works:



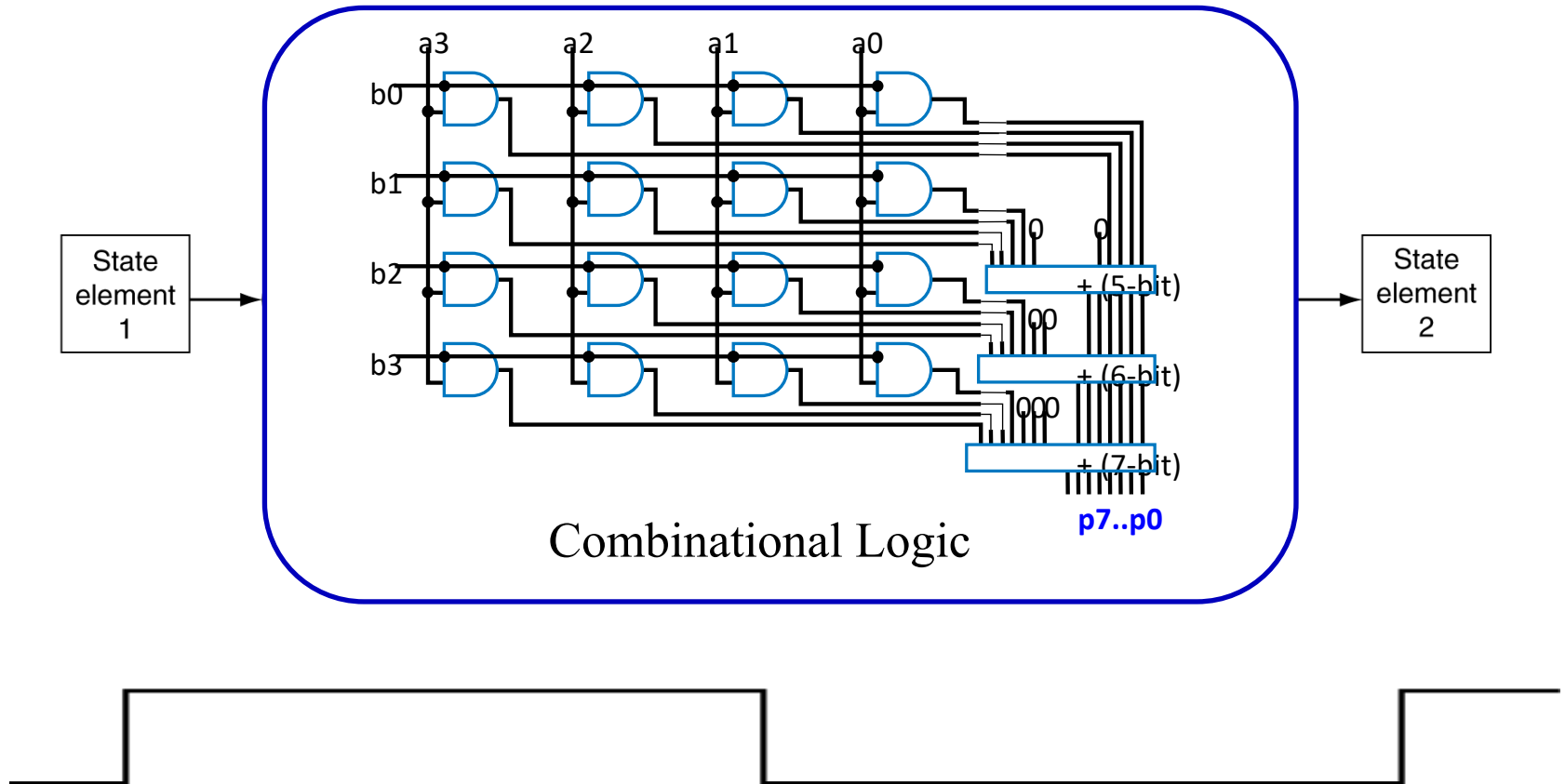
Clocking Methodology for FSM

- Combinational logic calculates data during clock cycles
 - Between clock edges
- Clock cycles should be
 - **Long enough** to allow combinational logic to complete computation
 - Longest delay determines clock period
 - **Short enough** to ensure acceptable performance and to capture small changes on external inputs



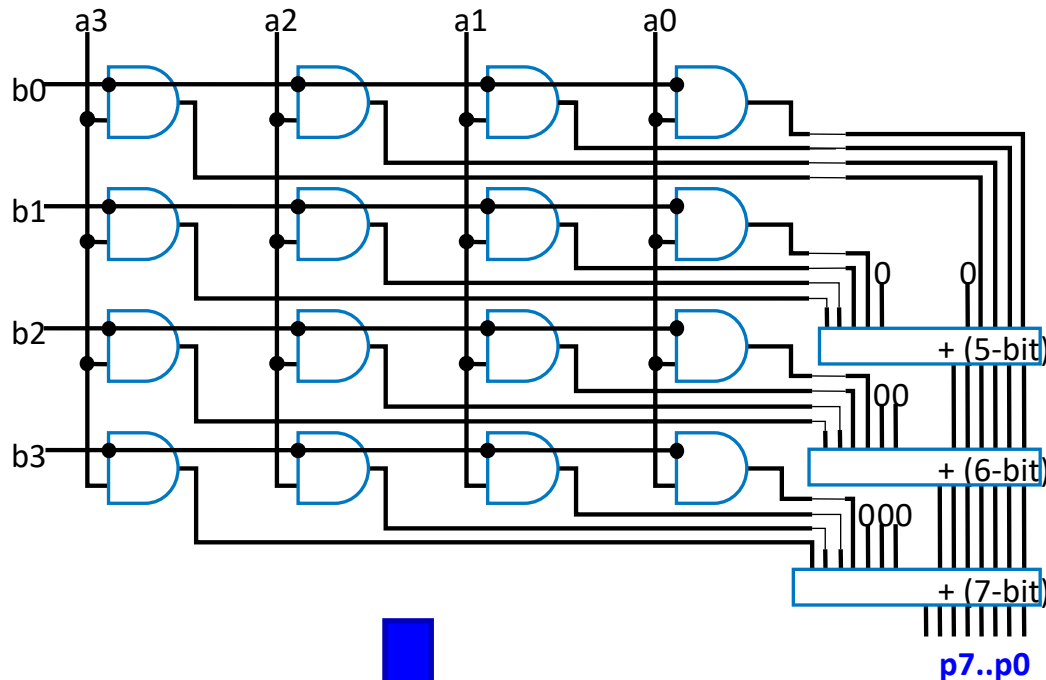
Clocking Methodology for FSM

- Big combinational logic circuit

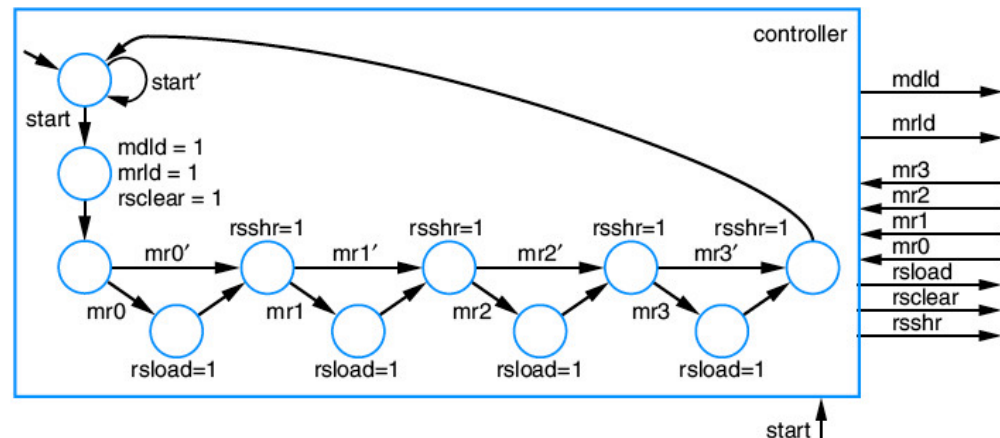


Clock cycle will be very long, clock signal will be slow

Multiplier – Array Style to FSM style



Recall how the array style of multiplier design was changed to State Machine style. Why?

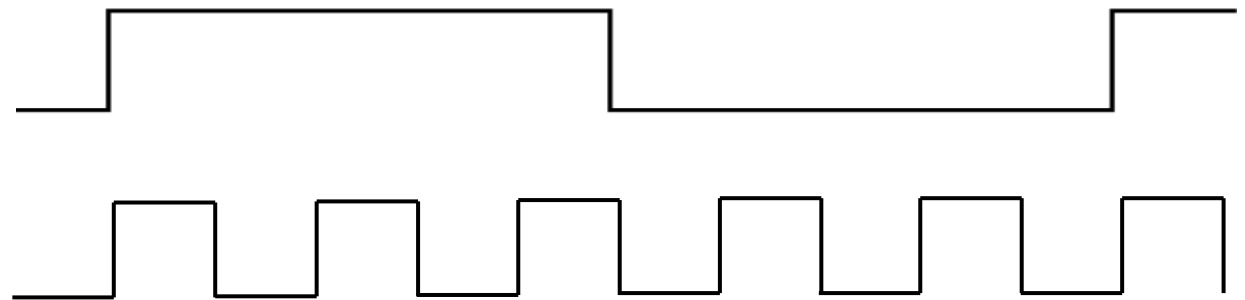


Multiplier – Array Style to FSM style

```

0110
0011
----
0110
0110
0000
+0000
-----
00010010
    
```

Complicated operations is divided into steps of smaller operations. So clock is faster.



Step 1	Step 2	Step 3	Step 4
0 1 1 0	0 1 1 0	0 1 1 0	0 1 1 0
* 0 0 1 1	* 0 0 1 1	* 0 0 1 1	* 0 0 1 1
-----	-----	-----	-----
0 0 0 0	0 0 1 1 0	0 1 0 0 1 0	0 0 1 0 0 1 0
	+ 0 1 1 0	+ 0 0 0 0	+ 0 0 0 0
	-----	-----	-----
	0 1 0 0 1 0	0 0 1 0 0 1 0	0 0 0 1 0 0 1 0

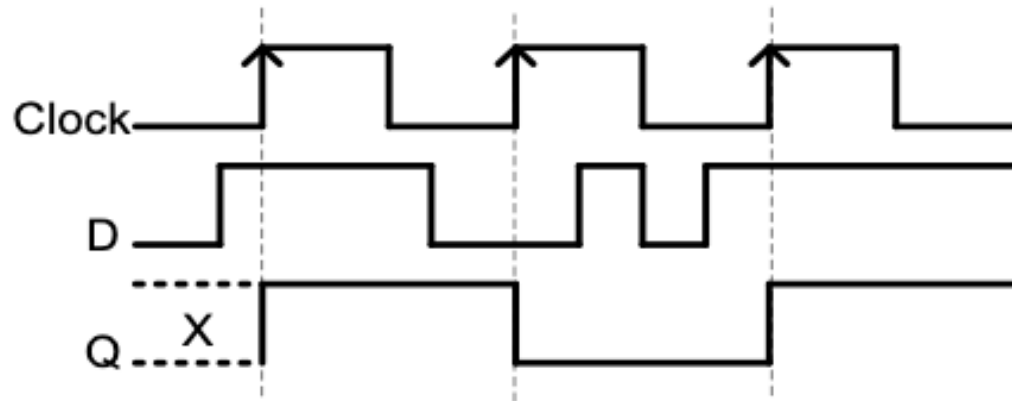
Summary

No matter how small a step is, we need to make sure it finishes its operations before the next clock edge comes. Otherwise...

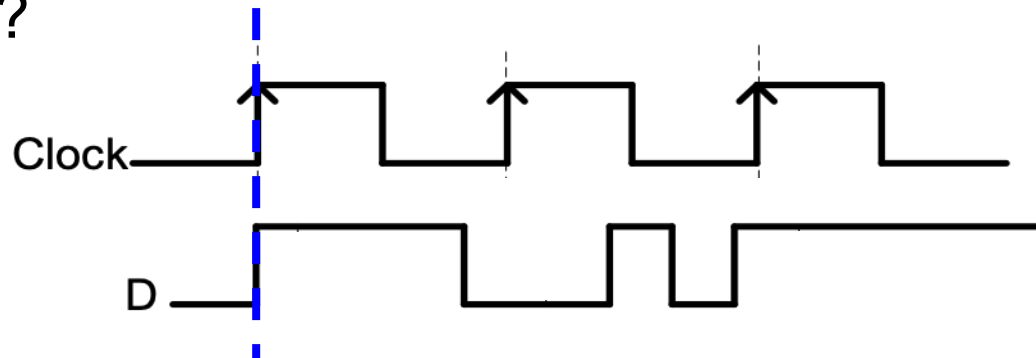


Rising-Edge Triggered D Flip Flop

- The output Q gets the value of input D at the time point of rising edge of clock



- What if...?

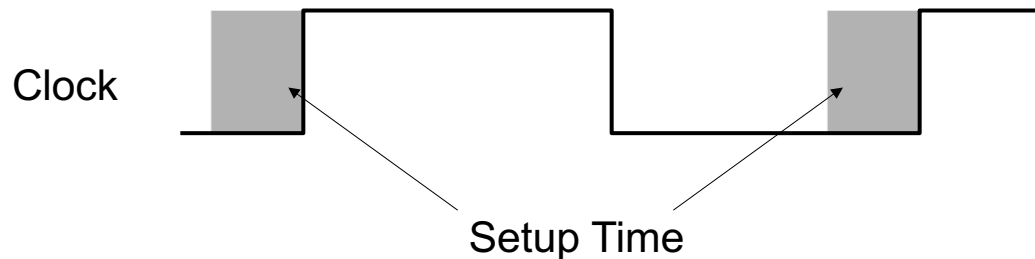


Outline

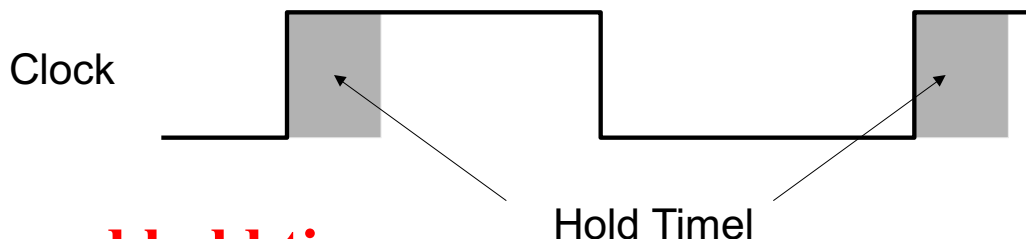
- Gated Clock
- Timing requirements for FSM
- Setup time and hold time
- Asynchronous input and synchronizer
- Switch debouncing
- Hazards

Hardware Constraints – Setup and Hold

- An edge-triggered flip-flop will not operate correctly if the data is not stable for a sufficient time before and after the clock edge
- Clock triggered elements may be put in nondeterministic state
- **Setup Time**
 - Minimum time that data must be stable prior to the triggering edge



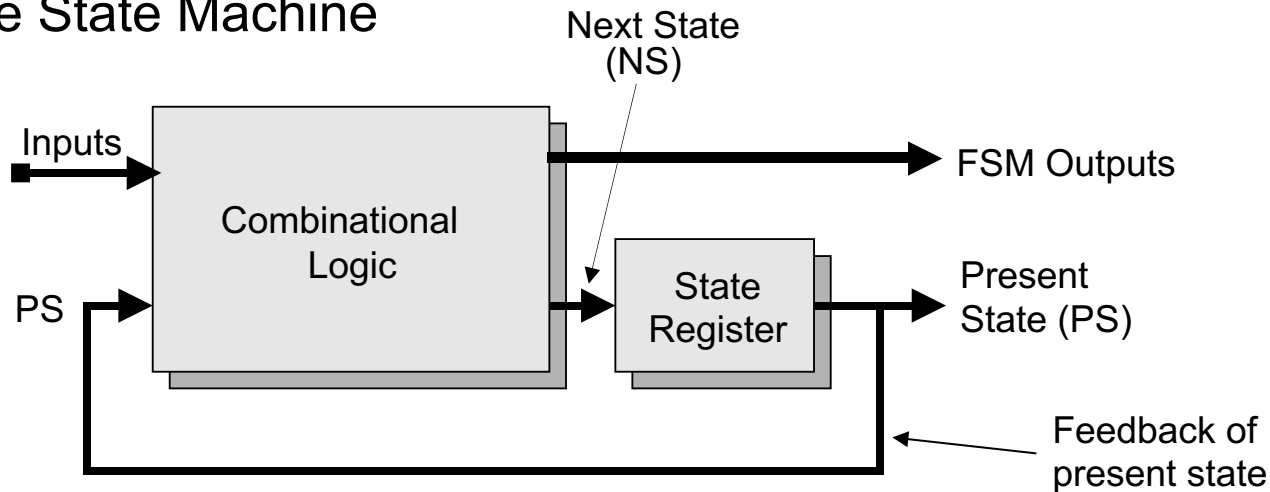
- **Hold Time:**
 - Minimum time that data must remain stable after the triggering edge



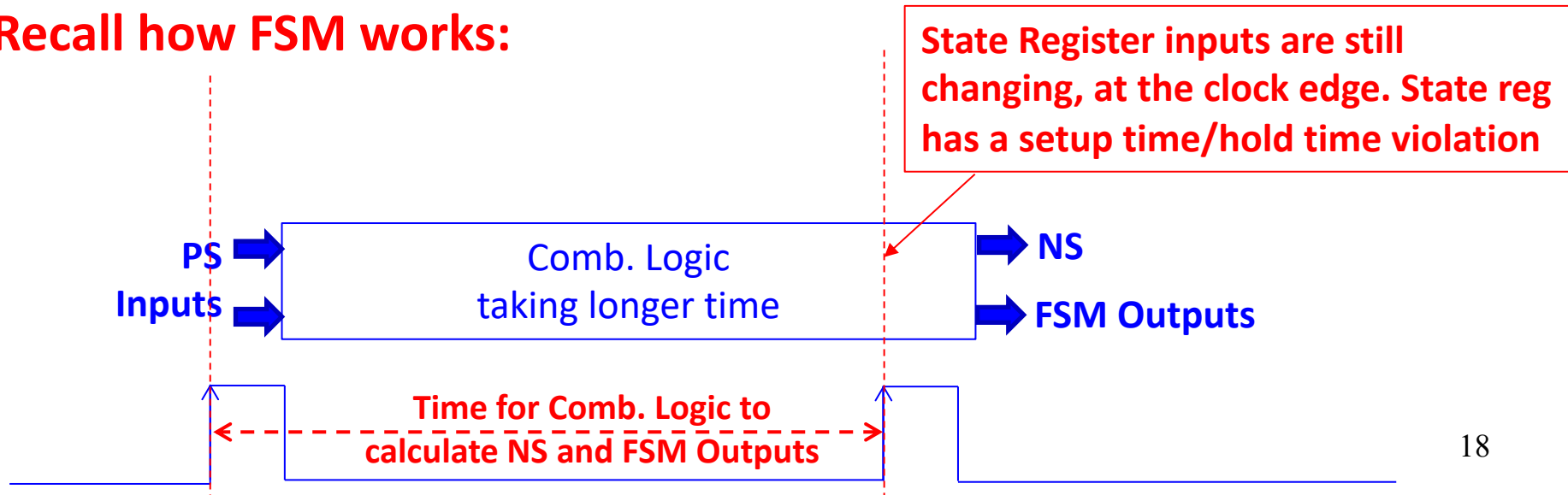
Typical setup and hold time: ns

State Machine

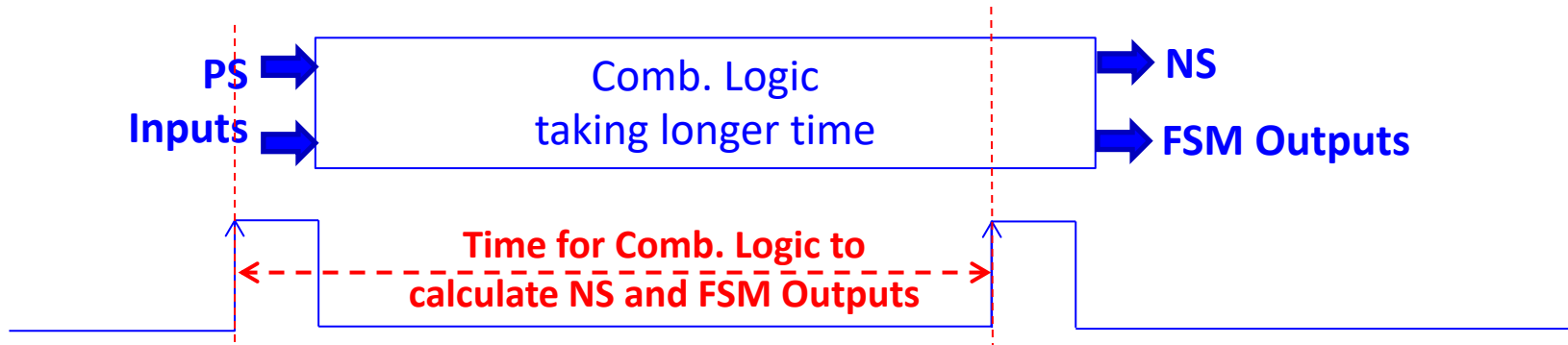
- Finite State Machine



Recall how FSM works:

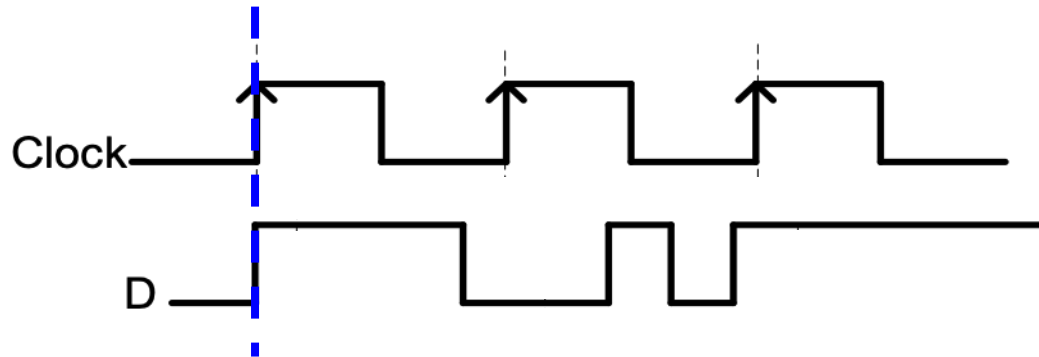


Setup/Hold Time Violation



- Setup/hold time violation may happen when the combinational logic takes too long time to finish
- To avoid the setup/hold time violations in FSM
 - Stretch clock cycle time
 - Optimize combinational circuit for shorter delay
 - Split combinational circuit into smaller steps, one step in a clock cycle (pipelining)

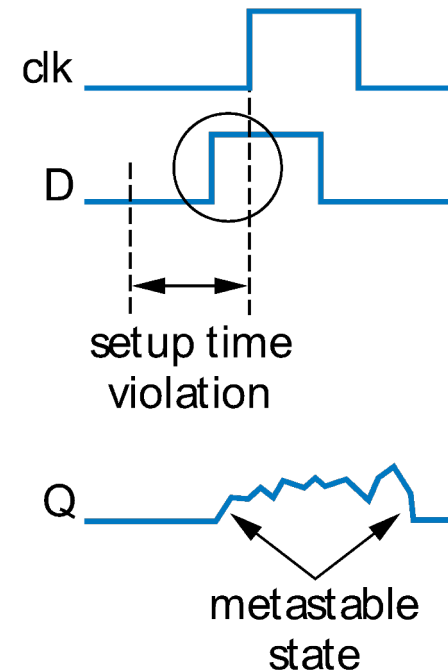
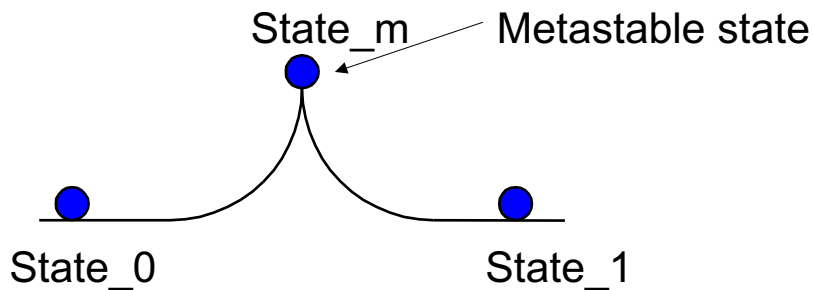
Setup/Hold Time Violation



- Setup/hold time violation may be caused by Asynchronous Input (an input that may come at any time)
- To avoid the violations
 - Use different flip-flops with smaller setup/hold times to reduce the likelihood of catching an input change
 - May not be completely avoided

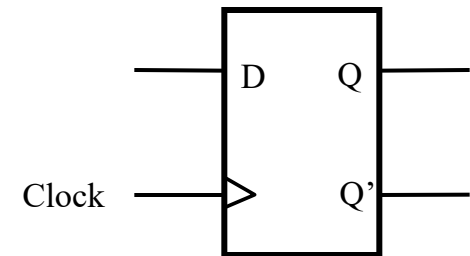
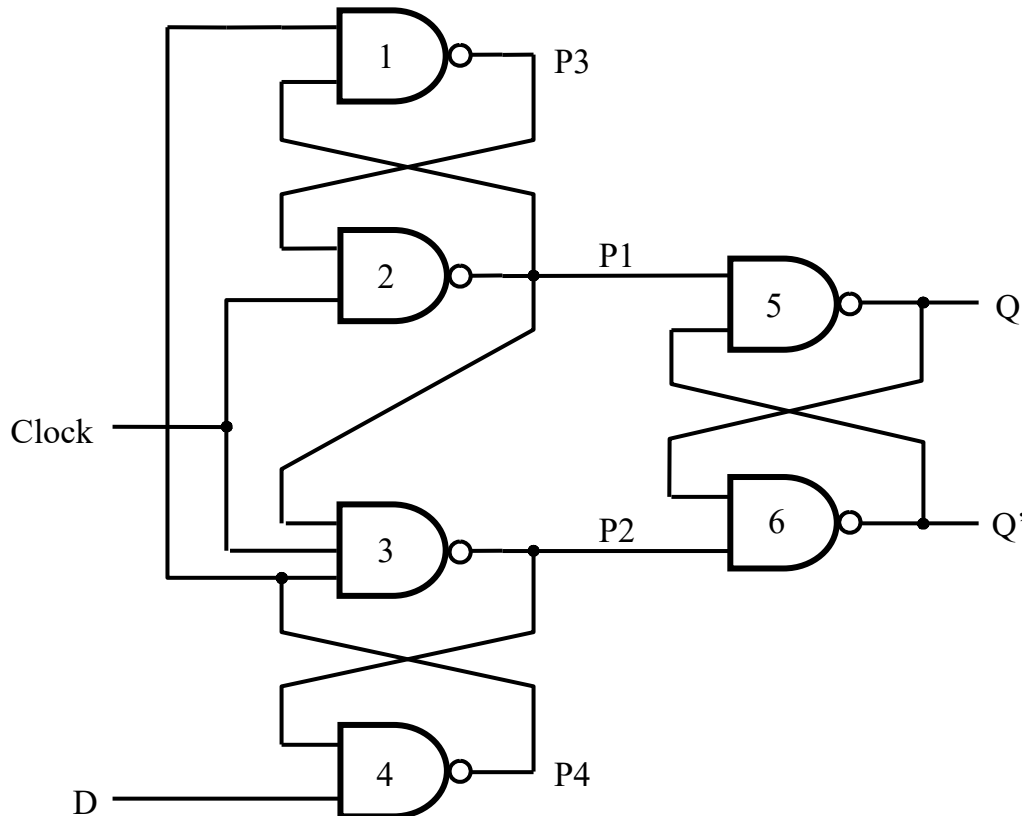
Metastable State

- When setup/hold time is violated, the flip-flop may enter a **metastable state**.
 - **Metastable state**: circuit outputs a signal other than stable 1 or 0
 - The output eventually settles to 1 or 0, **but we don't know when and which value**



Metastable State in a DFF

- Assuming $D = 0 \rightarrow 1$ while rising edge of clock ($0 \rightarrow 1$)
 - Outputs will oscillate and may never stop
 - In reality, DFF outputs will settle eventually after some time (hopefully within a clock cycle)



Outline

- Gated Clock
- Timing requirements for FSM
- Setup time and hold time
- Asynchronous input and synchronizer
- Switch debouncing
- Hazards

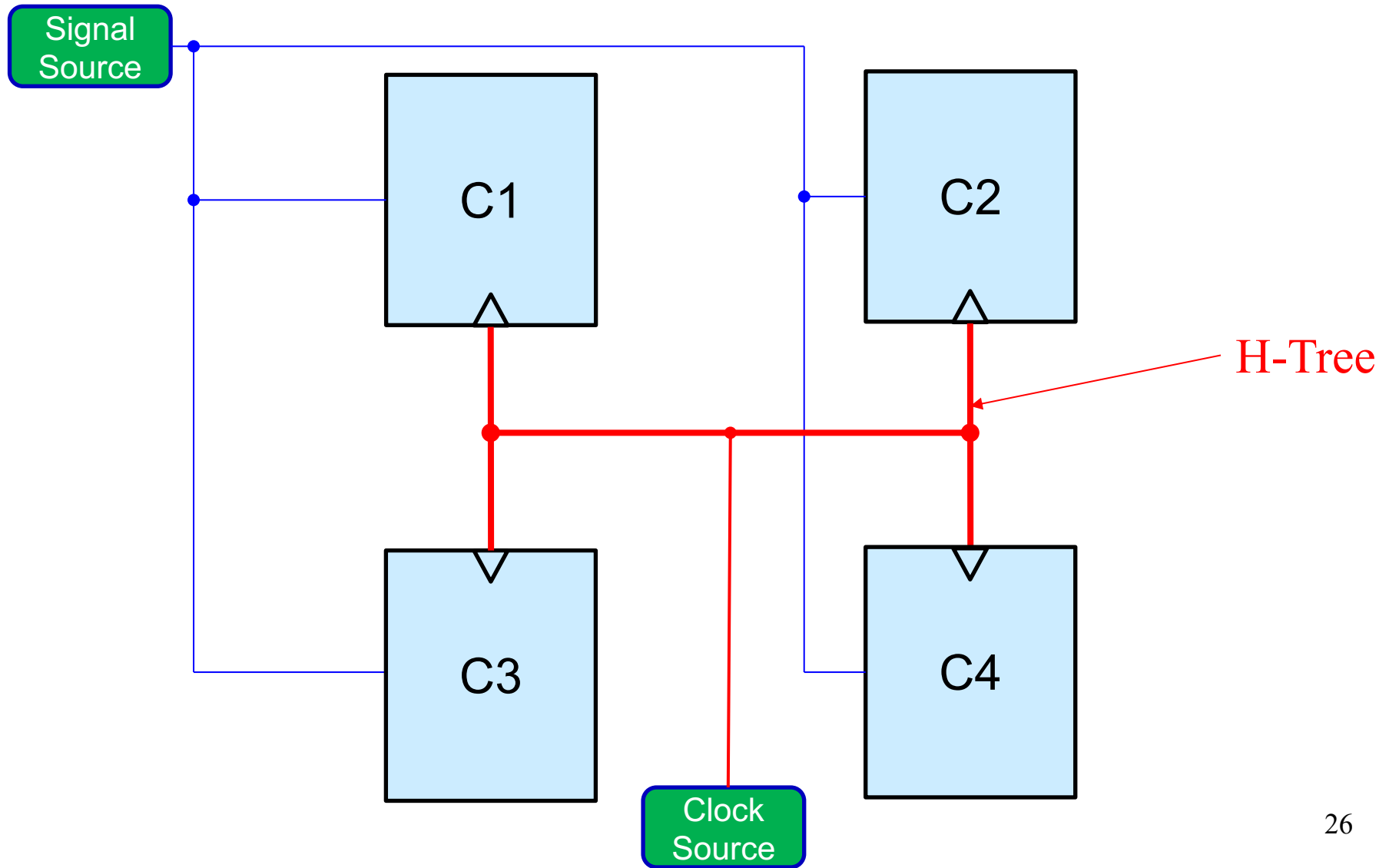
Asynchronous Inputs

- Inputs that may change independent of clock
 - Examples: Keyboard, push buttons, sensor outputs from the environment
- Synchronous system may be driven by asynchronous inputs which may put flip flops into the metastable state

Problems and Solution

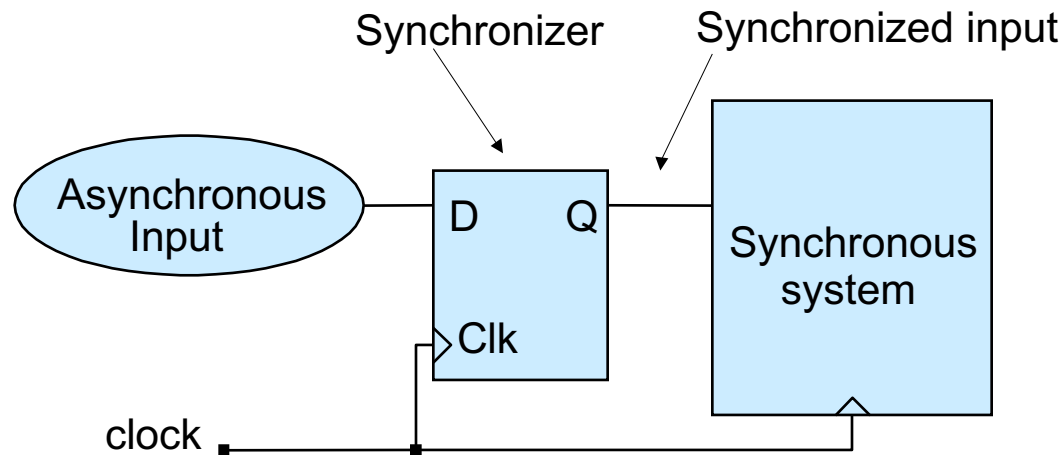
- Problems caused by asynchronous inputs
 - The **unpredictable arrival of an asynchronous input** may cause a setup/hold condition to be violated
 - **Different fanout path delays** might cause asynchronous arrival of the same signal that is "caught" by some devices and "missed" by others
- **General rule:** Ensure that all inputs are synchronized to the clock signal
- Reset and set can be exceptions

Different Fanout Path Delays



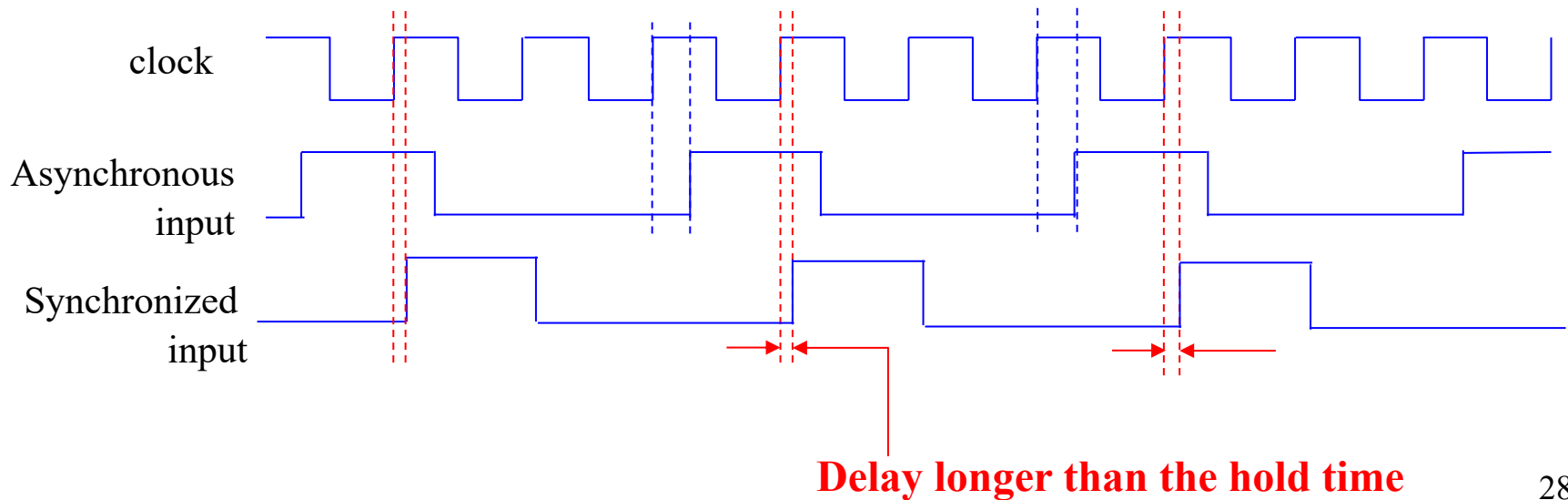
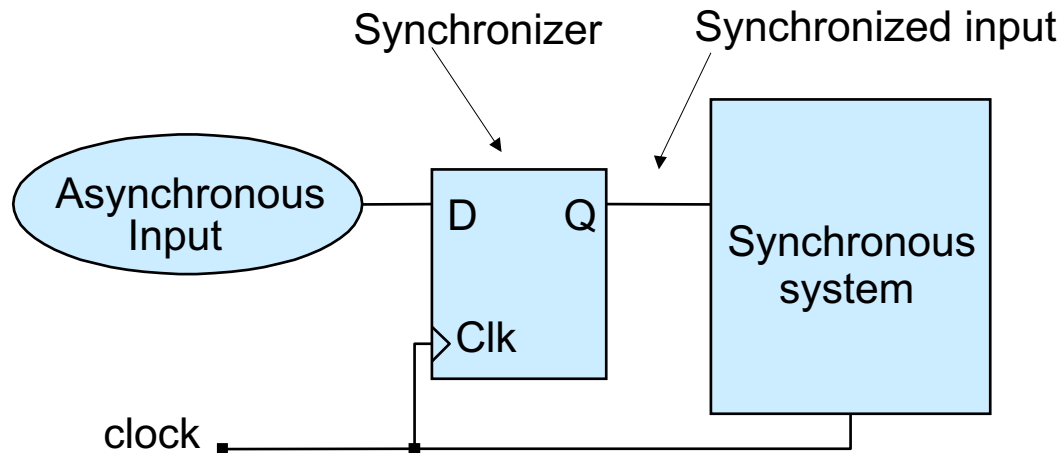
Synchronizer For Unpredictable Arrival of Inputs

- Use a D-type flip flop to synchronize the asynchronous input



- If there is any change on the synchronized input, it will happen after setup time and hold time of the clock edge due to delay of the DFF. So, the Synchronous System won't be put in metastable state.

Synchronizer

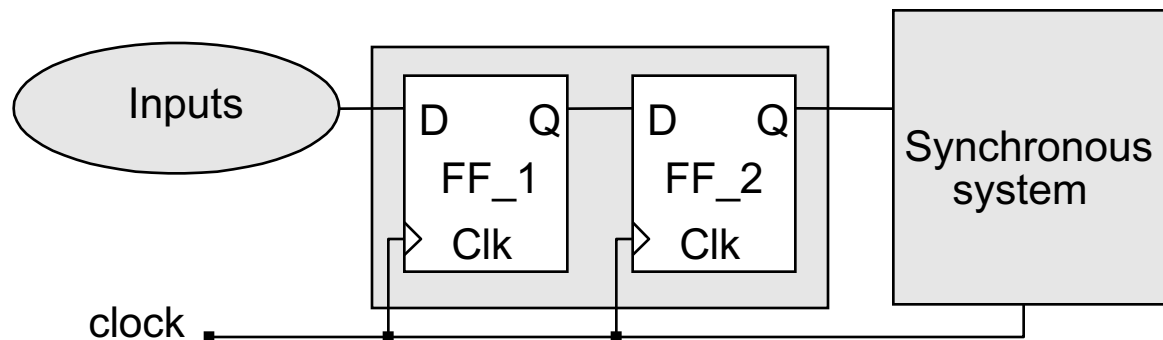


Synchronizer Failure

- The systems input is synchronized, but the setup and hold times of the synchronizer may be violated
 - The synchronizing flip-flop may enter a metastable state and disrupt system operation.
- The flip-flop may be in metastable state, then its state will be nondeterministic, its output may be interpreted as
 - “0”, or
 - “1”, or
 - enter a metastable state, output is not synchronized anymore
- A circuit that uses the output of a synchronizer that is in the metastable state is said to experience a **synchronizer failure**.
- To recover from a synchronizer failure
 - wait for the circuit to leave the metastable state, the time is unpredictable, or
 - execute a reset to put the system into a known state.

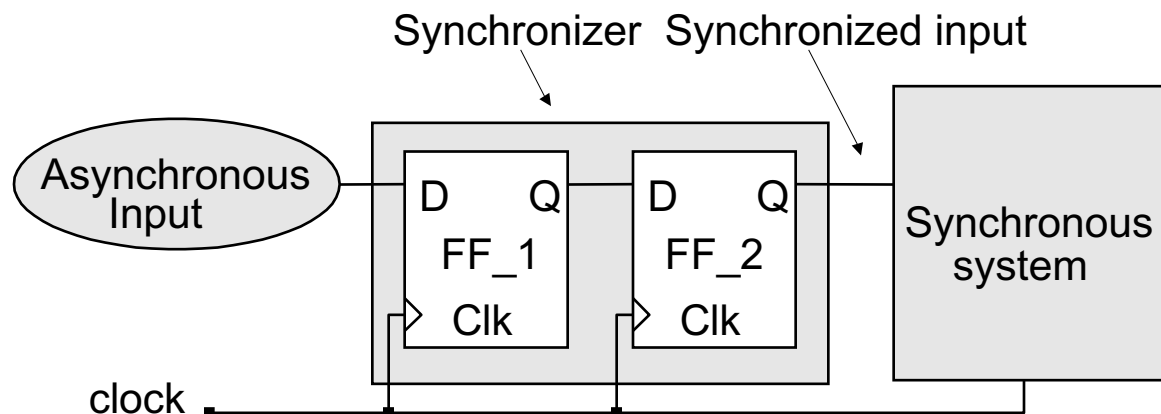
Reduce Possibility of Synchronizer Failure

- Use flip-flops having the **shortest possible setup and hold** intervals
- **stretch the clock period** to allow more time for the circuit to recover
 - Output finally settles in 0 or 1
 - But means slower circuit
- Insert **double-synchronizing D flip-flops** after the asynchronous inputs



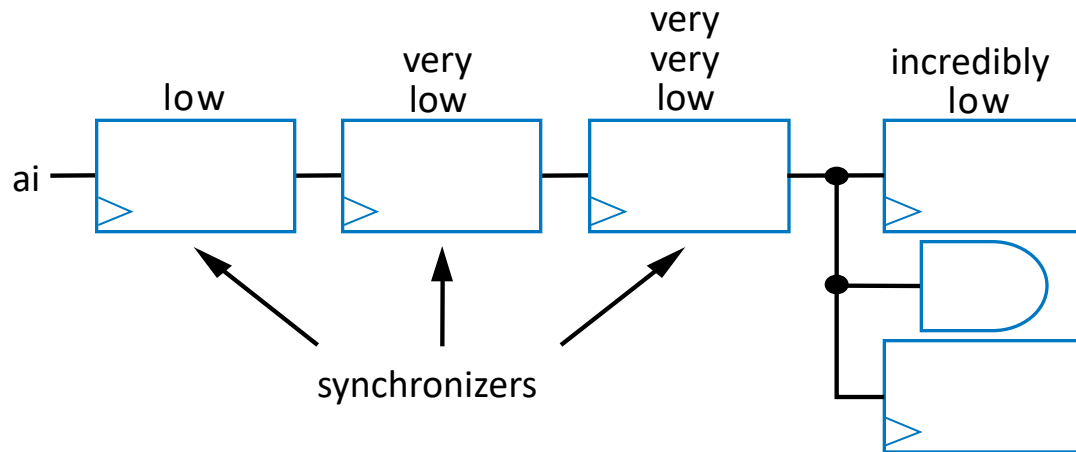
Double DFF Synchronizer

- The first flip-flop might enter a metastable state, but **has a clock period to recover**
- **Both flip-flop** must enter a metastable state to have a synchronizer failure
- It is **less likely that FF_2 will see an input that is not valid**, thus it is less likely that FF_2 will enter a metastable state
 - Typically, 1 clock cycle is long enough for 1st DFF to recover
 - Output of 2nd DFF is synchronized and stable, but **unpredictable**



Synchronizer with More FFs

- More FFs, lower possibility of Synchronizer Failure, but unnecessary

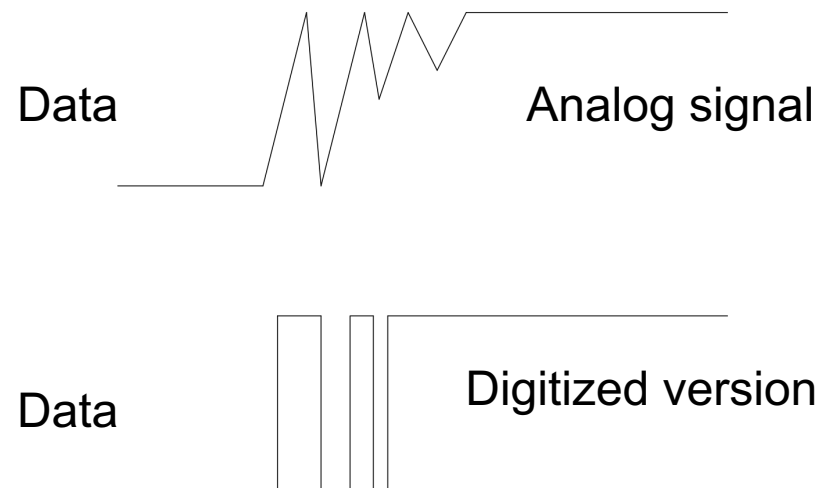
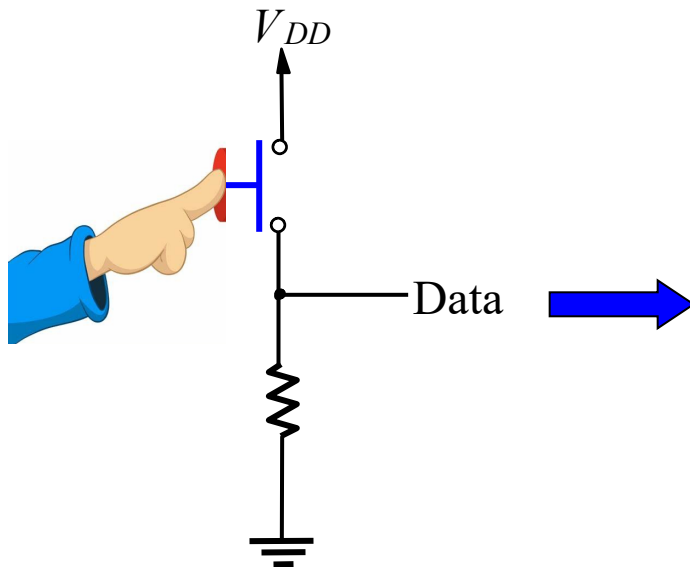
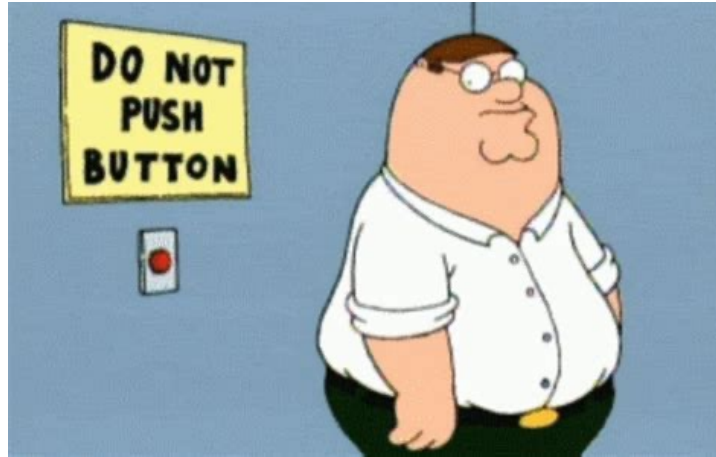


- Synchronizer failure can never be 100% avoided
 - We can just maximize the Mean Time Between Failures (MTBF) -- a number often given along with a circuit

Outline

- Gated Clock
- Timing requirements for FSM
- Setup time and hold time
- Asynchronous input and synchronizer
- Switch debouncing
- Hazards

Push Button



Bouncing Switch

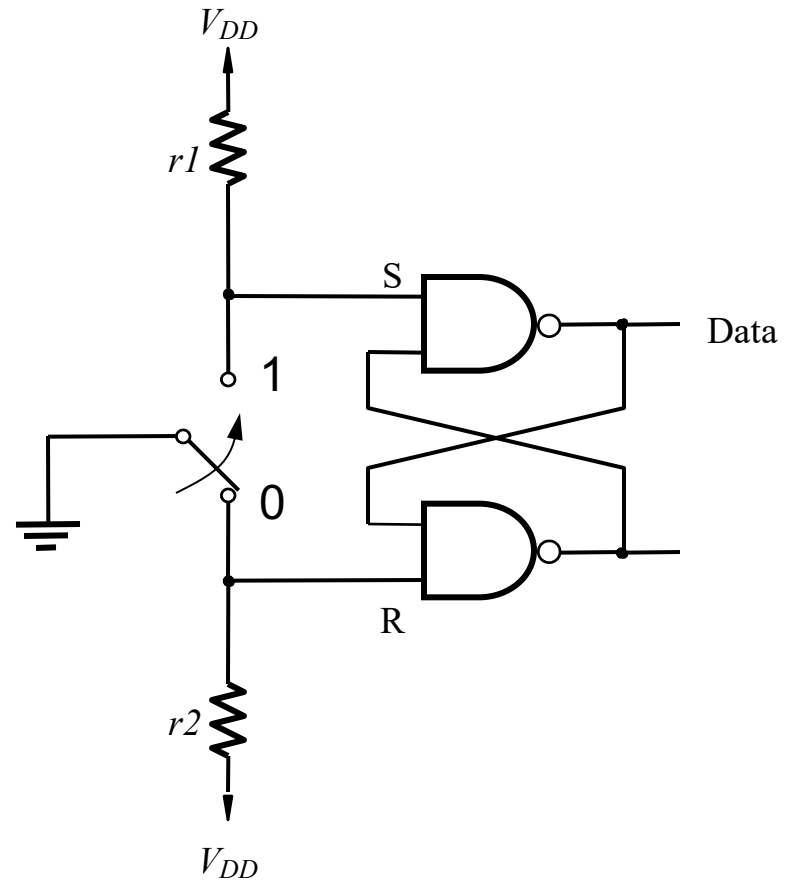
- Asynchronous input may be produced by a mechanical switch or a push button or this sort of mechanical components
- When changing from one position to another, the switch may bounce away from the contact point

Switch Debouncing

- Debouncing circuit: SR latch
- When throw is at position 0,
S = 1, R = 0, Data = 0
- When throw changes to position 1
R = 1, S = (0 ↔ 1), Data = 1
- Data is latched regardless of S
assuming switch won't bounce
back to position 0 and touch it

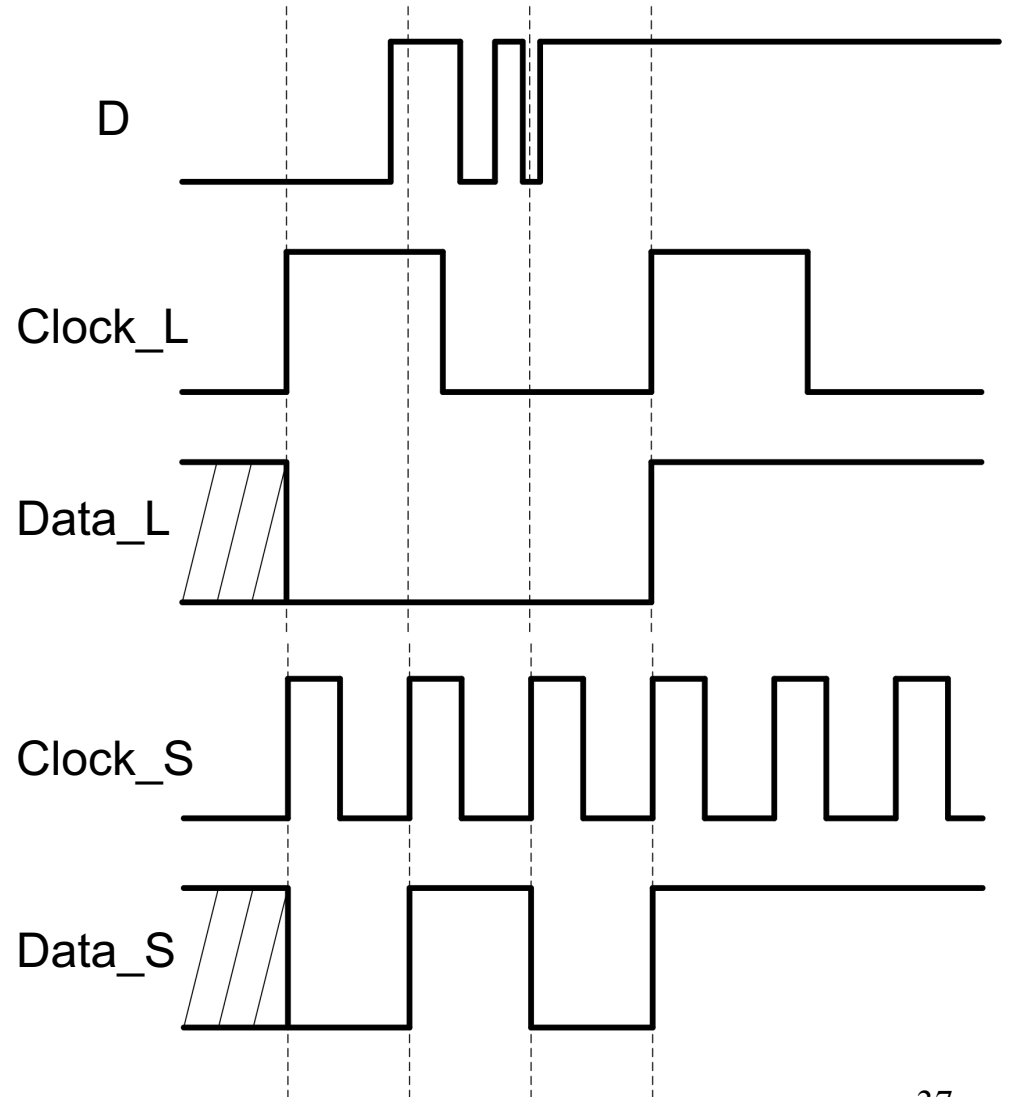
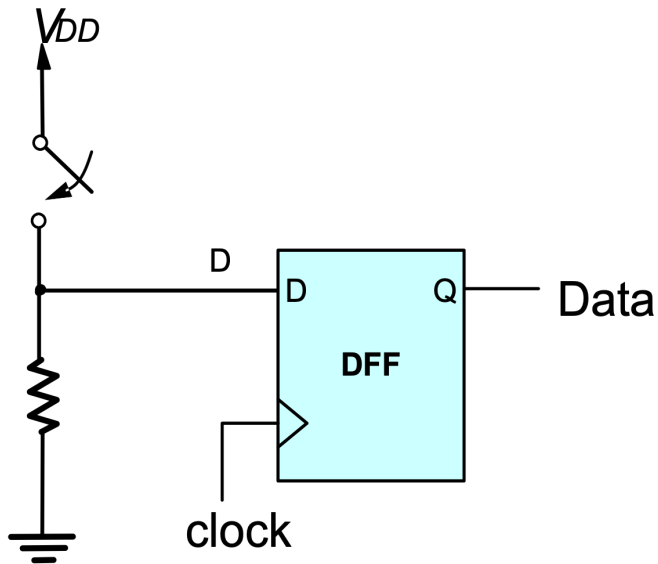
S	R	Q	Q ⁺
0	0	0	X
0	0	1	X
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

not allowed
set
reset
hold



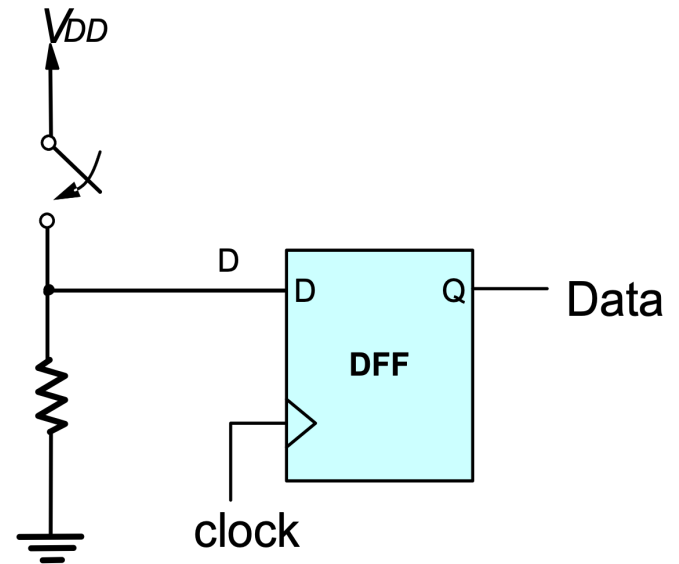
Switch Debouncing

- D Flip Flop may be used for switch debouncing
- But clock needs careful design



Switch Debouncing

- The clock signal triggering the D flip flop should be slow enough to allow the switch to settle
- The clock should be fast enough to allow the asynchronous input to take effect as soon as possible
- **Bouncing time: ms**
- The DFF may be replaced by a double DFF structure
 - Serving as both synchronizer and debouncing circuit



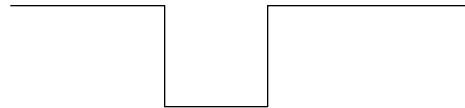
Outline

- Gated Clock
- Timing requirements for FSM
- Setup time and hold time
- Asynchronous input and synchronizer
- Switch debouncing
- Hazards

Glitches and Static Hazards

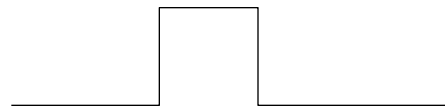
- The output of a combinational circuit may make a transition even though the patterns applied at its inputs do not imply a change. These unwanted transitions are called **glitches**.
- A circuit in which a glitch may occur is said to have a hazard.
- static 1-hazard

1-Hazard



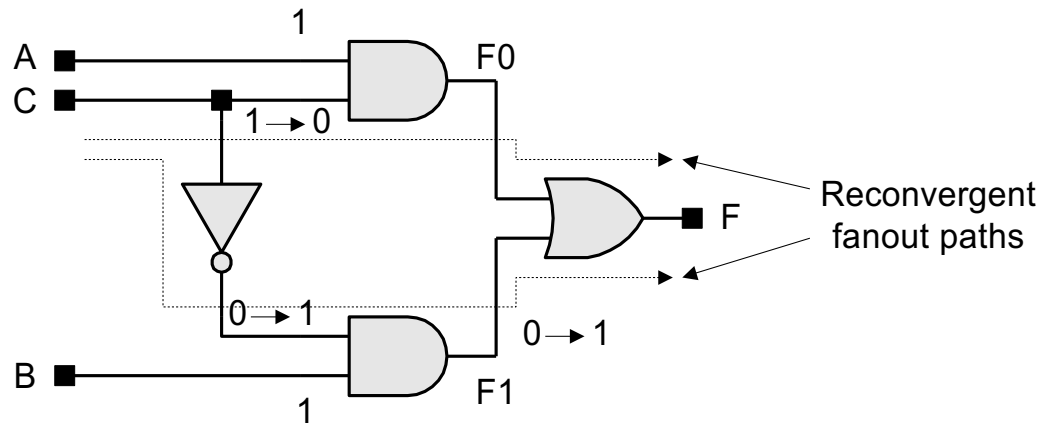
- static 0-hazard

0-Hazard



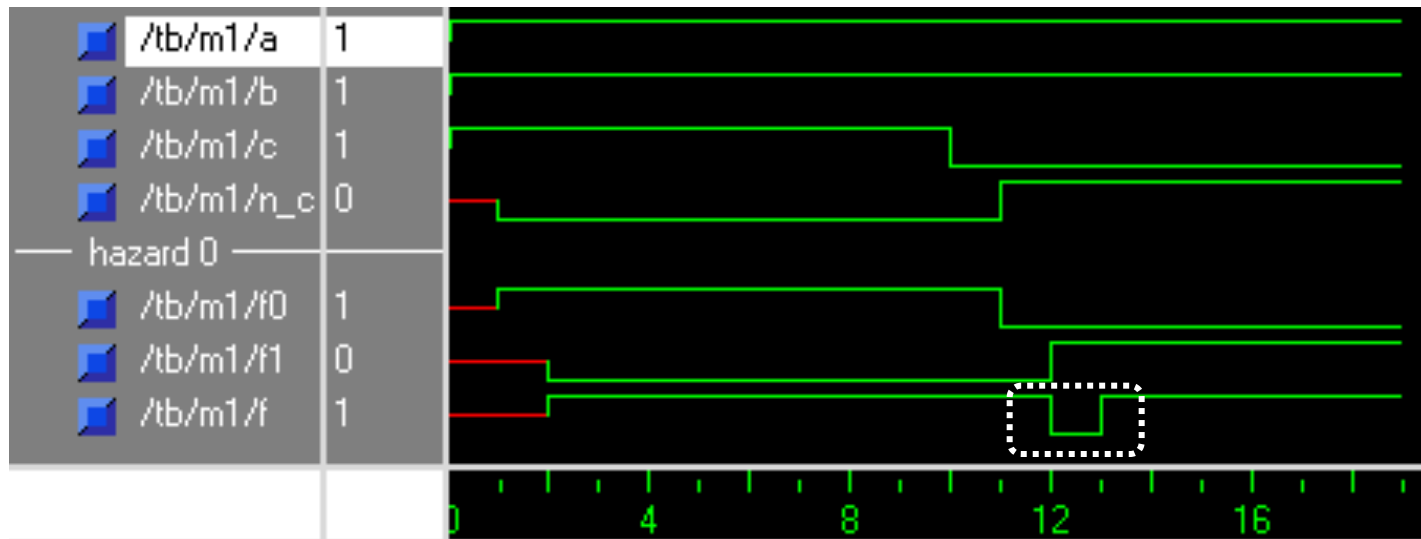
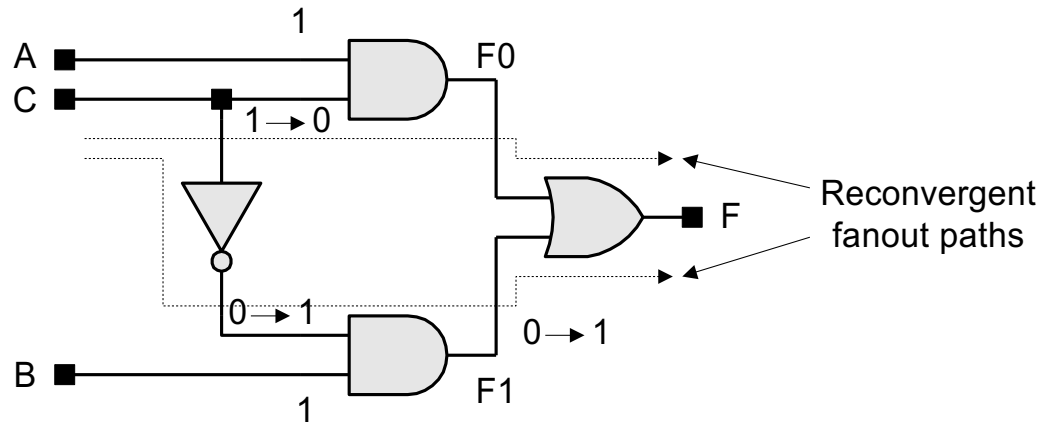
Static Hazards

- Static hazards are caused by different propagation delays on the **reconvergent fanout** paths
- Example: $F = AC + BC'$



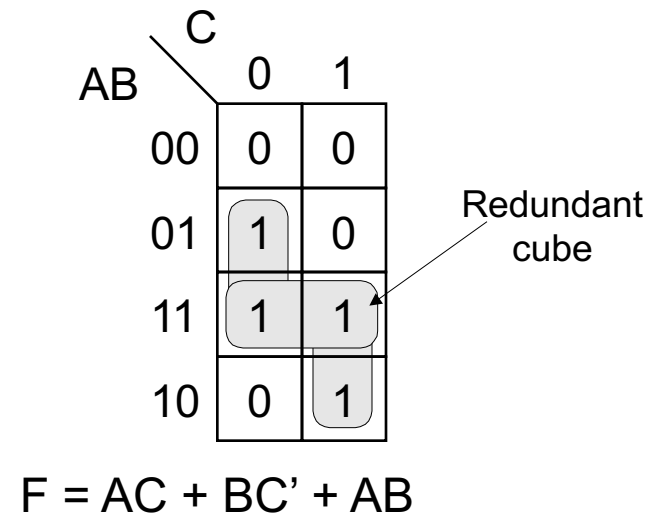
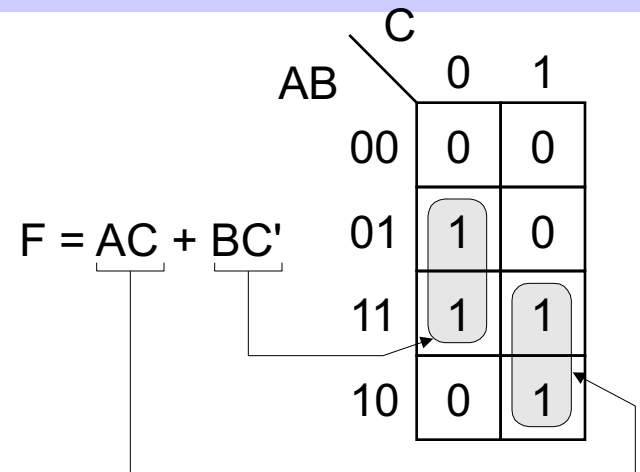
- Initial inputs: $A = 1$, $B = 1$, $C = 1$ and $F = 1$
- New inputs: $A = 1$, $B = 1$, $C = 0$ and $F = 1$
- With non-zero propagation delays, the path to F1 will be longer than the path to F0, causing a change in C to reach F1 later than it reaches F0.

Static Hazards Example



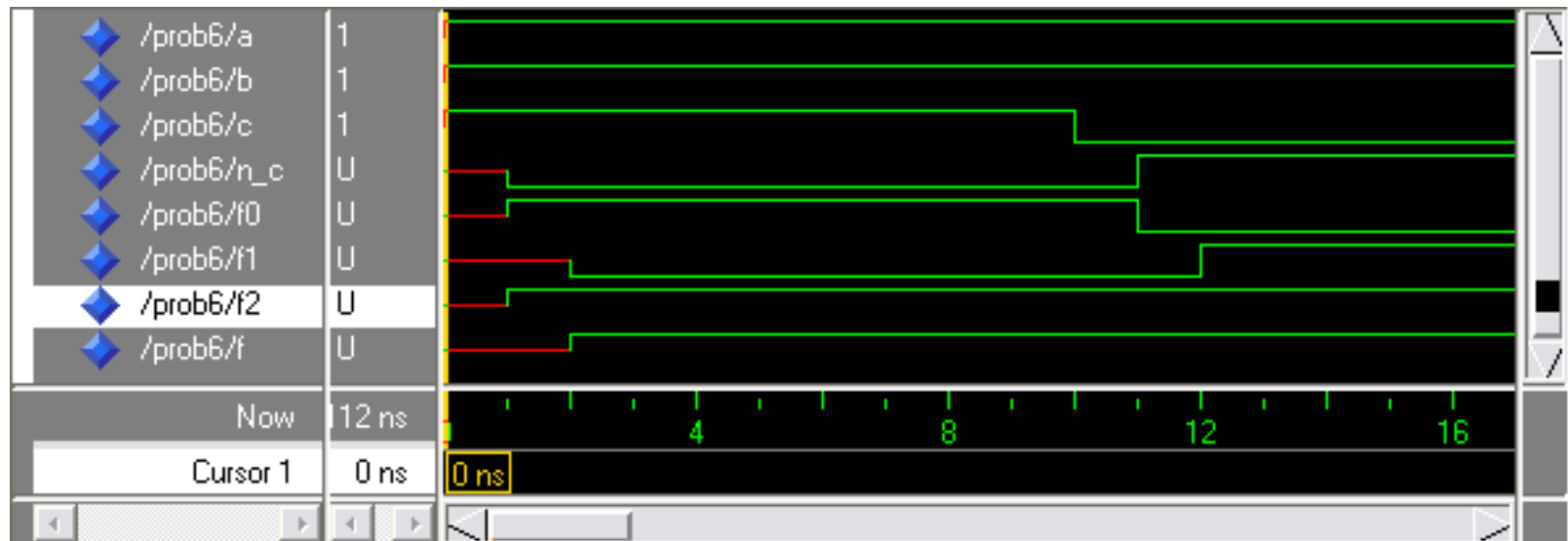
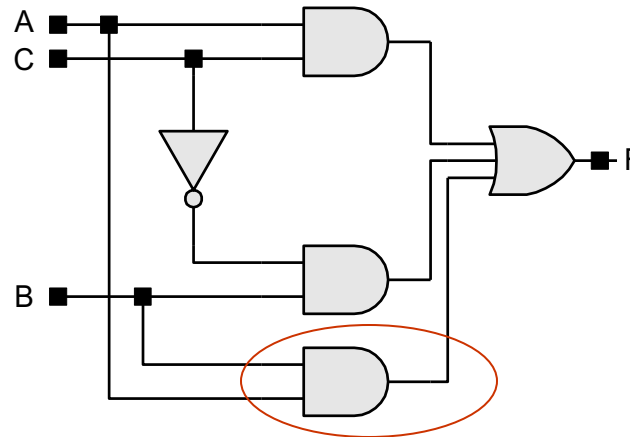
Static Hazards

- Hazards might not be significant in a synchronous sequential circuit if the clock period can be stretched
- A hazard is problematic in an asynchronous circuit (including Mealy FSM)
- If transition is within the same group in the K-map, hazard won't occur
 - will occur when transition from one group to another
- **Hazard Removal**
A static hazard can be removed by using a redundant PI



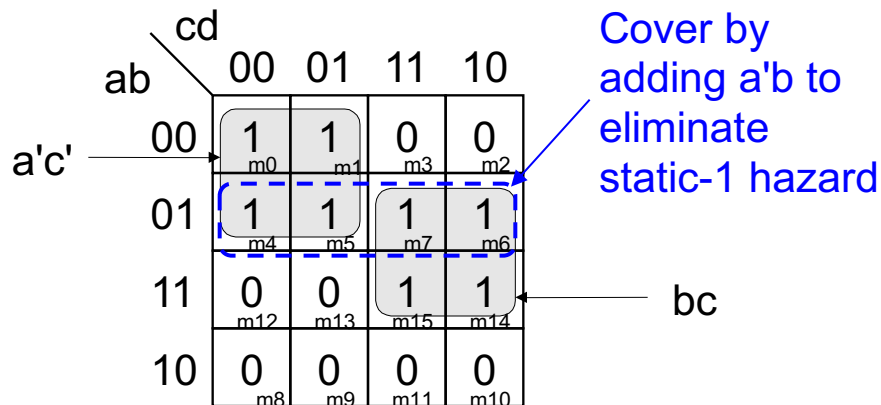
Static Hazards Example

Static 1-Hazard-Free circuit

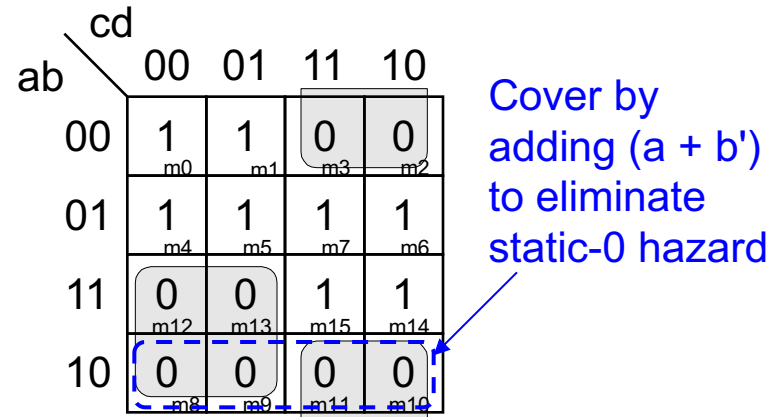


Static Hazards

- Static 0 hazard might not be removed in a static 1 hazard free circuit
- To eliminate static 0 hazard:
 - method 1: cover the adjacent 0s in the corresponding POS expression
 - method 2: first eliminate the static 1 hazards. Then verify in the K-map if 0 hazards have been eliminated already



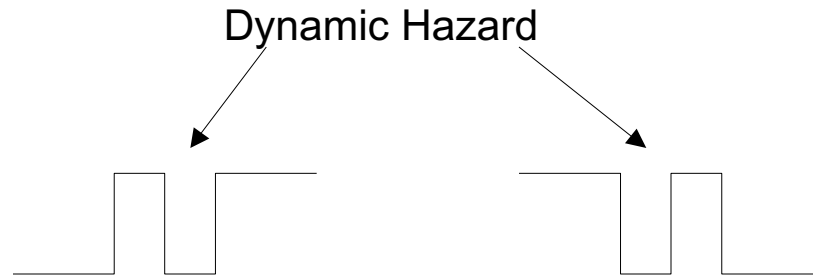
Eliminating static 1 hazard
 $F = a'c' + bc + a'b$



Eliminating static 0 hazard
 $F = (a' + c)(b + c')(a' + b)$
 $= a'c' + bc + a'b$

Dynamic Hazards

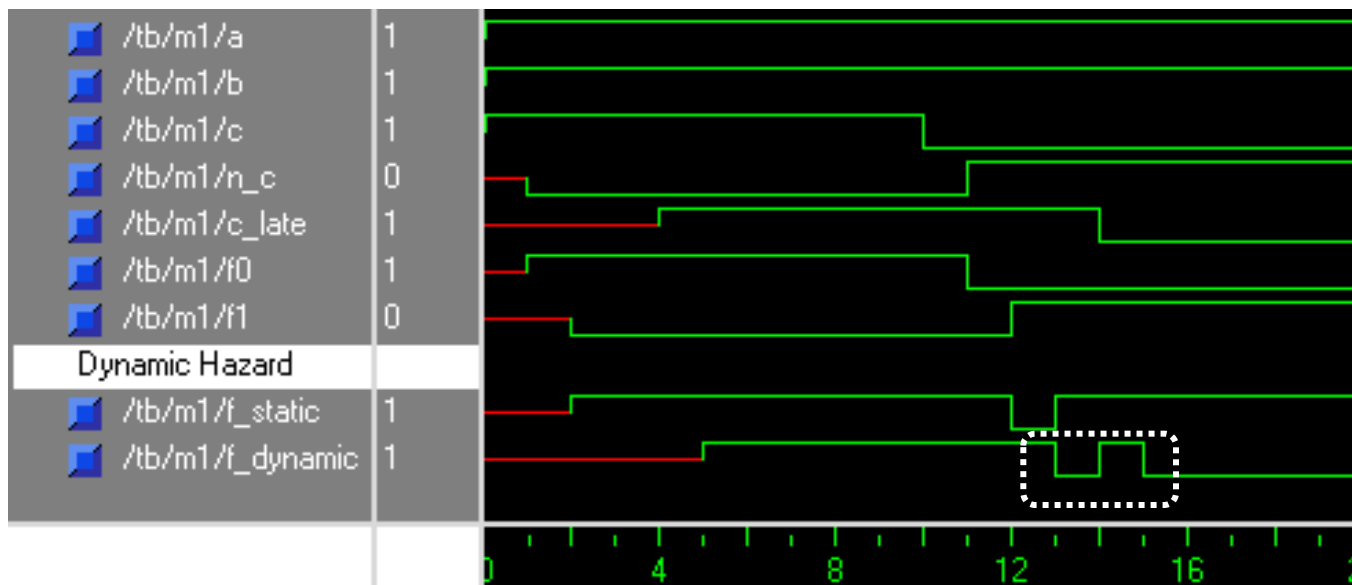
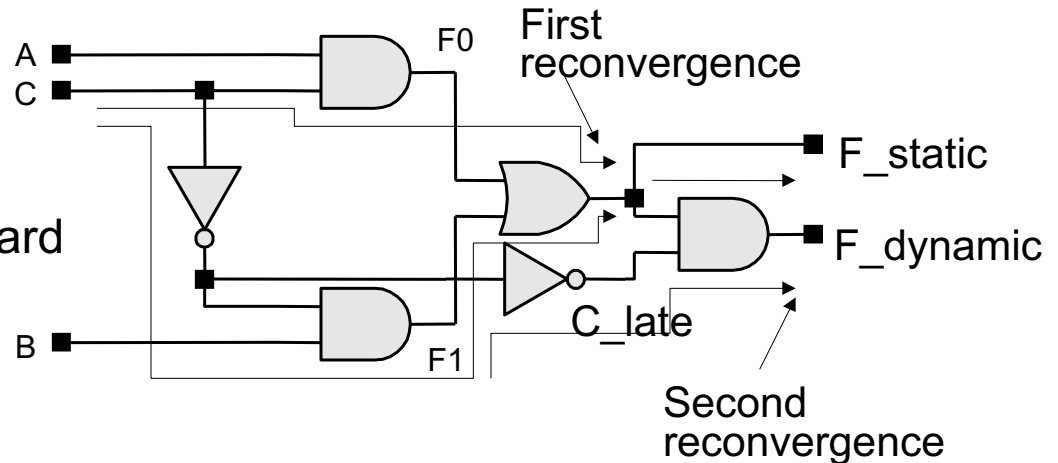
- Dynamic Hazard: output changes more than once as a result of a single output change



- Dynamic hazards are a consequence of multiple static hazards caused by multiple reconvergent paths in a multi-level circuit.
- **Hazard Removal:**
 - Elimination of all static hazards eliminates dynamic hazards.

Dynamic Hazards Example

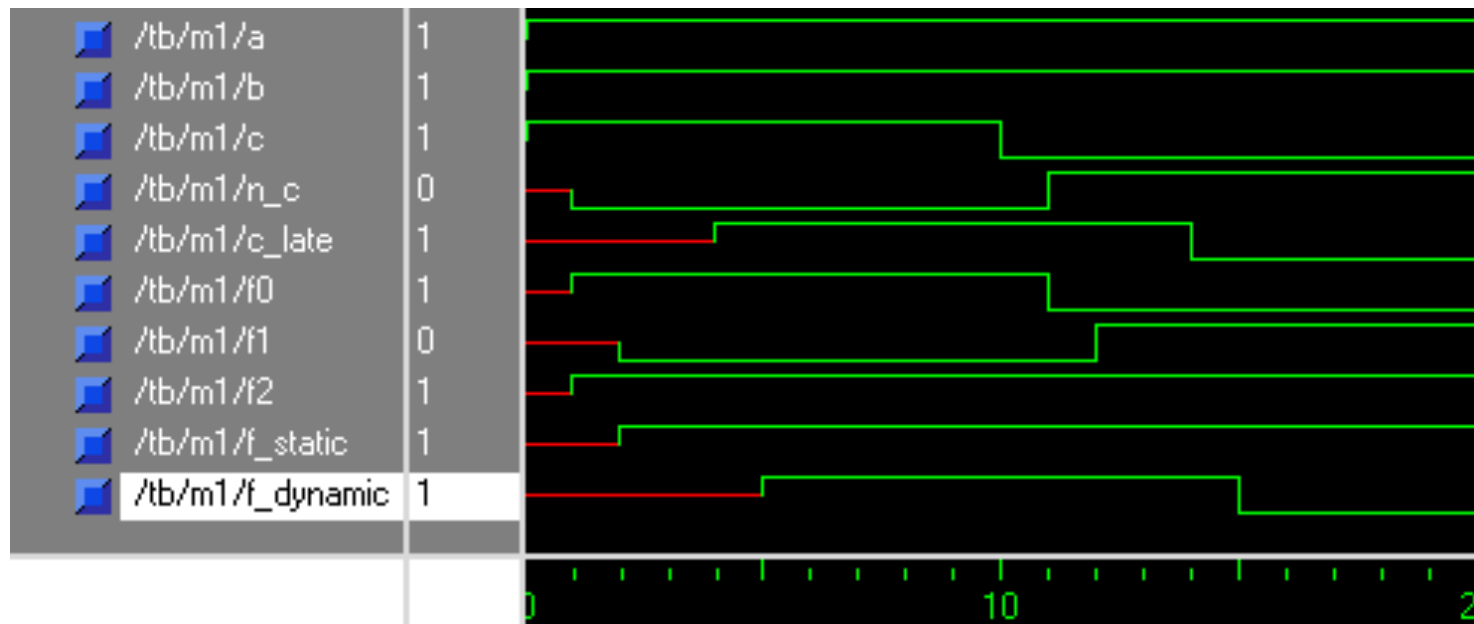
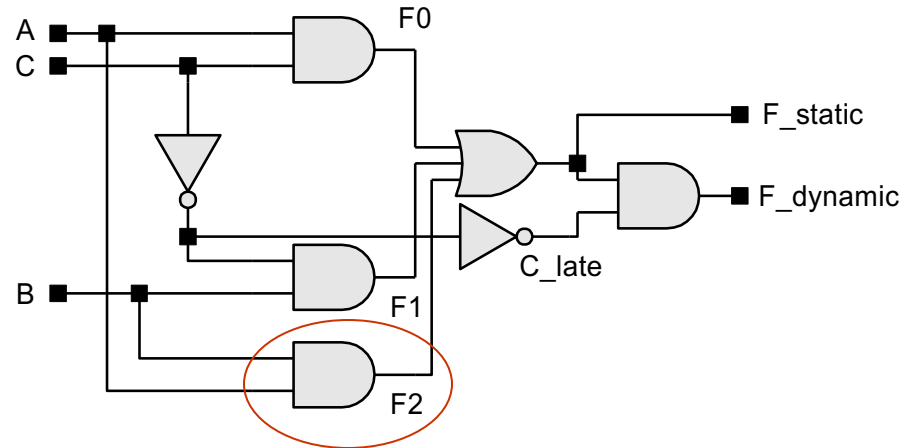
F_static has a static hazard
F_dynamic has a dynamic hazard



Dynamic Hazards Example

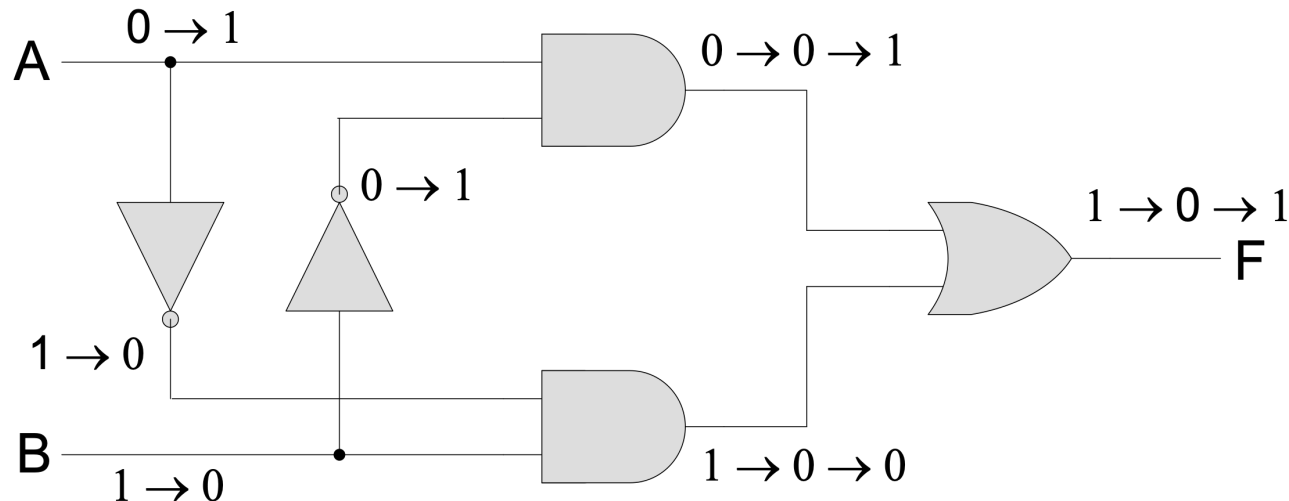
		C	
		0	1
AB	00	0	0
	01	1	0
	11	1	1
	10	0	1

Redundant group
Eliminates both
1-hazard and
0-hazard

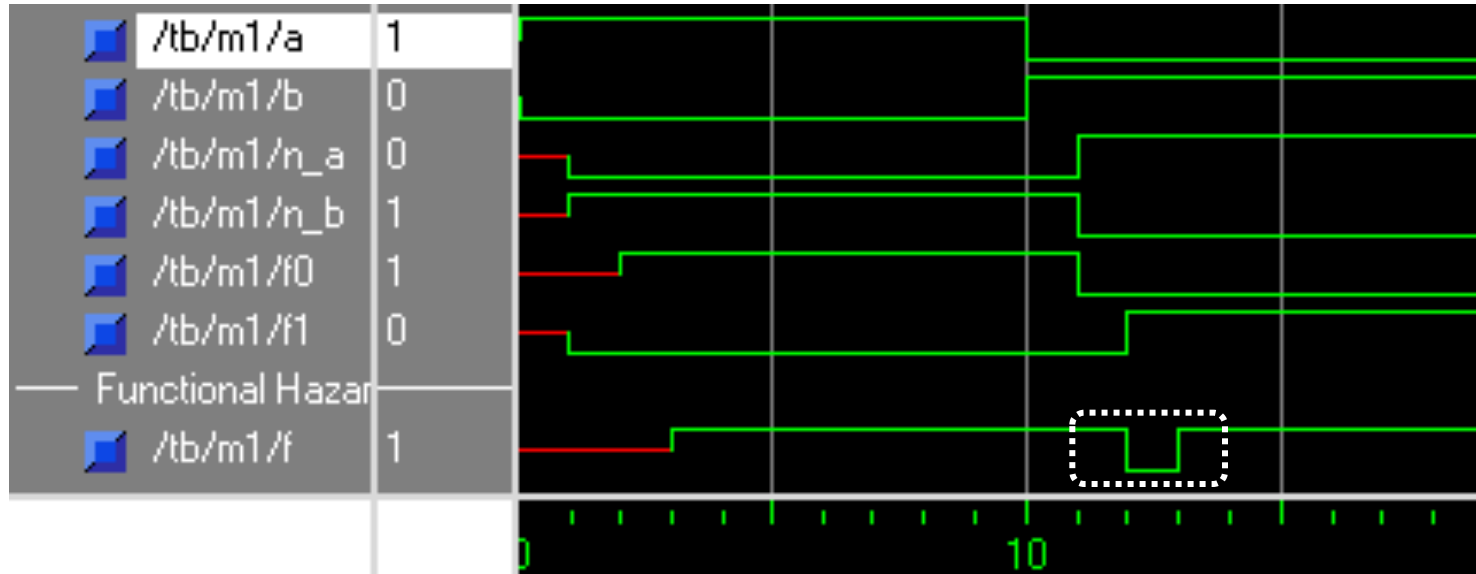


Functional Hazard

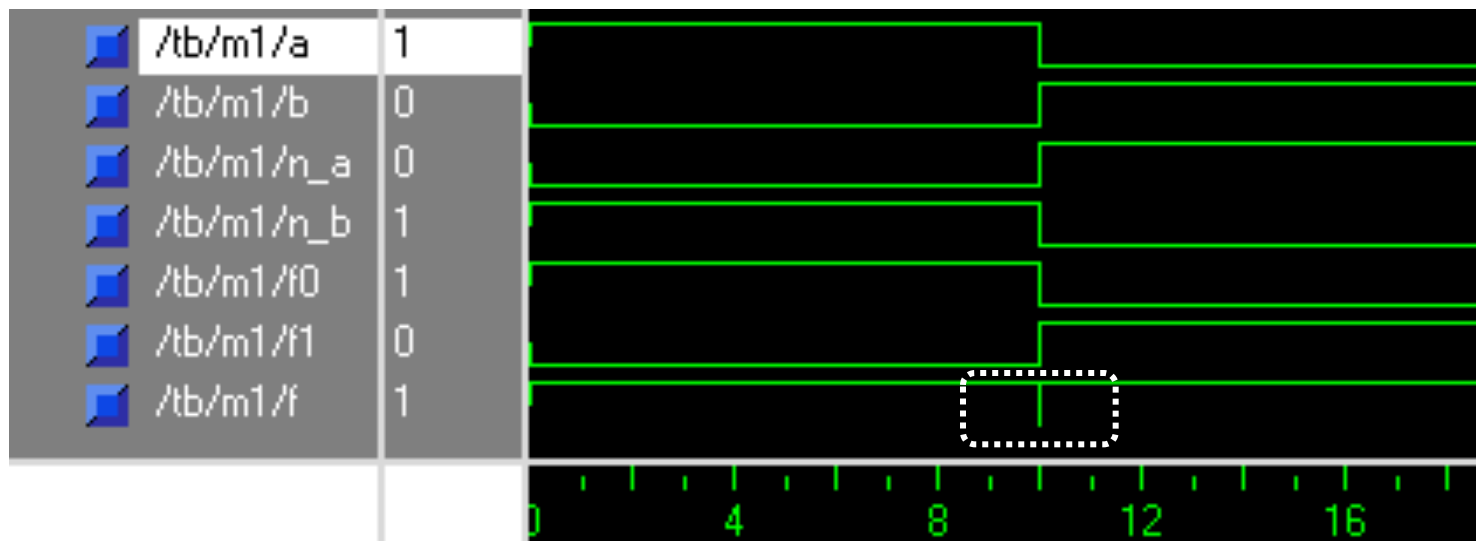
- Solutions to both static hazard and dynamic hazard are based on the assumption that only one input is changing at a time
- When more than one input changes occur at the same time, the hazard caused is more complicated to remove
- Example:



Functional Hazard



Simulation
with delay



Simulation
without delay.
Many sim
tools show
this to warn
for functional
hazard

Functional Hazard

- Functional hazards are unsolvable hazards which occur when more than one input variable changes at the same time
- Hazards such as functional hazards can not be logically eliminated as the problem lies with actual specification of the circuit
- K-map view of the functional hazard

